Measured figures behind the code

Every number in `tsp/` that came from a measurement rather than a specification, with the conditions it was taken under. Comments in `tsp/` cite the headings here instead of carrying the derivations, so a constant can be re-derived without reading the module it lives in.

Firmware behaviour the manuals contradict or omit is in `UNDOCUMENTED.md`. This file is the app's own timings, geometry and arithmetic.

Instrument: DMM6500, firmware 1.7.17a, unless stated.

Reading a sample out of a reading buffer

`buf.readings` is a firmware proxy, so every index is an IPC round trip. Three independent measurements agree:

figure	how it was taken
21.73 us	the read component of a 20 000-sample decode step, timed per phase
22.75 us	derived from the 0.455 s a capture that `acq_copy_out` spends
22.82 us	timed directly, 20 000 reads in one loop

Independent of buffer style, so the cost is the round trip and not the payload: `STYLE_FULL` stores more per reading and reads within 0.07 % of `STYLE_STANDARD` (22.836 us against 22.820, three interleaved runs each).

Acquisition is the opposite -- 20 000 samples take 207.96 ms into a `STYLE_STANDARD` buffer and 221.65 ms into `STYLE_FULL`, +6.58 % at 8.3 sd. So payload size costs time where it is written and is invisible where it is read. `buffer.make` (1.8 ms) and `buf.clear()` (0.17 ms) are style-independent.

Streaming by real-time extraction is not possible

9600 baud at 10 samples/bit needs 96 000 samples a second: ~10 us a read with nothing left for decoding, ~5 us to leave half the budget. 21.73 us is 2.2x and 4.3x over those. Double buffering cannot help, the deficit being throughput rather than a gap between captures, so the app decodes one window at a time.

Decode phase timings

Timed per phase with `tools/bench_priming.py`, 9600 baud, W = 20 000:

phase	time	what it is
levels	0.958 s	the decimated pass, `ck_level_n` samples
reading	0.530 s	the contiguous lead-in read
edges	0.101 s	`sig_edges` + `sig_idle`
format	1.043 s	the format search, the most expensive per sample
window	0.685 s	one decode slice, x20

Every phase is proportional to W, so one per-sample cost bounds all five, which is what makes the latency bound real. Timing only `sig_edges` and `ua_run` over a window already in a Lua table understates the total threefold, because the buffer read is part of the cost.

ck_smp_us = 73

The format phase scales with FRAMES, and samples-per-bit varies 2:1. At W = 8653: 0.451 s at 9600 baud (8.33 sa/bit) against 0.634 s at 250 kBd (4.0 sa/bit). Two points give 32.5 us/sample + 1.63 ms/frame, so 52 us/sample at 8.33 sa/bit and 73 at 4.0. The worst is used: one always-safe constant beats two that must be evaluated before the format is known. Costs ~30 % window width at low rates -- throughput, not correctness.

ck_win_us = 48

One 20 000-sample decode step is 0.685 s, i.e. 34.25 us a sample, of which 21.73 is the buffer read and does not scale with frames. Only that one operating point was timed, so the worst case comes two ways: scaling the whole figure as the format phase scales from 8.33 to 4 sa/bit (x 73/52) gives 48, and holding the read fixed while doubling the frame-proportional remainder gives 47. The higher is used.

ck_slice_n = 1

Samples per press = W x n, and n = floor(budget / (W x ck_win_us)), so raising n by narrowing W cannot buy more than budget / ck_win_us samples per press: **9375** at 0.45 s. `ck_win_for()` takes the smaller of two bounds (`sdec.ck_win_for()`), and the one that binds today is the other one -- budget / ck_smp_us = 0.45 / 73 us = **6164**, not the ck_win_us bound at all. That 1.52x is the whole prize and taking it needs W <= 4687, i.e. n = 2. Three constraints then do not fit at 0.5 s -- n = 2, a W that stays a useful width (>= 5000, asserted in `test_serial.lua`), and the 0.9 margin. Any two hold: n = 2 with W = 5000 spends 0.480 s of the 0.5 s bound, inside it but on 4 % margin rather than 10 %.

Why a slice is bounded in windows

`while timer.gettime() < budget` is the obvious way to spend a latency budget, and it is wrong here.

There is **one timer** on this instrument -- `timer.cleartime()` / `gettime()` is global -- and a budget check inside the decode loop would have to reset it per slice, which collides with both of the things that legitimately hold it:

holder	what it holds the timer for
`record_run()`	clears it at the start of a flow-controlled run and reads it between windows, as the run-wide wall-clock bound (`fc_maxsec`). A slice clearing the timer would reset that bound every window and the run would never reach it. The decode is *inside* the interval being measured, being the larger half of a window.
`strm_absorb_arm()`	stamps the moment a run ended, to decide whether a queued press is still absorbable. A slice clearing the timer after that stamp would make a press made seconds later look immediate.

`stream_acquire()` does **not** hold it, stated because it is easy to assume otherwise: it bounds itself by summing its poll period into a counter and never touches the timer, so `strm_maxsec` is not a timer user. The timer is free *between* windows -- which is what lets `record_run()` bound a whole run -- and is never free inside the decode loop.

A window count is also deterministic, so the offline suite can assert exactly where a slice ends instead of depending on how fast the machine running the tests happens to be.

Why ck_job is resumable

Stream mode decodes up to 32 kB, and at the measured **881 byte/s** that is over half a minute inside one button handler. Nothing on this instrument can interrupt that: a press is not dispatched while Lua runs, presses queue, and `display.waitevent()` blocks indefinitely. The only thing that lets a press through is the handler **returning** -- so a long decode has to be a sequence of short ones, and something has to hold the middle of it.

That something is a job. It owns the three pieces that cannot be rebuilt per slice:

- the **sink** closure, which holds an open file handle and a partial 16-byte row in its carry. Recreating it per slice would reopen the file and drop the carry.
- the **tail** ring, whose contents and total-pushed counter are the run's, not a slice's.
- the `ck_decode` **state** -- window cursor, seam, running totals.

plus the format and the window table, primed once and shared by every slice. Creating a job costs nothing: priming is deferred to the first step, so the press that creates one is charged for one phase rather than all of them.

Window geometry

Overlap

`ua_run` stops when a frame runs off the end of its array, so a window ending at W leaves at most one partial frame, whose start t0 satisfies t0 > W - framebits*T. The next window must therefore begin at or before W - framebits*T, giving `overlap >= framebits * T` as the floor.

Two further frames plus `maxback` buy two things. `sig_cross` walks back up to `maxback` samples to find a crossing and that walk is truncated at a window's leading edge, so the first edge or two can be mispositioned; with margin they sit in already-emitted territory and `tmin` discards them. The seam also falls mid-overlap rather than at its rim, so de-duplication never decides a near-tie.

At 8.68 samples/bit and 10-bit frames: ceil(260.4) + 64 = **325 samples, 1.6 %** of a 20 000-sample window.

The seam: one carried number does continuation and de-duplication

After each window the absolute position at which the last emitted frame ended is known: it is t0 + (framebits - 0.5)*T, the same expression `ua_run` uses internally to stop a data transition inside a byte from being read as the next start bit. Handing that to the next window as `tmin0` makes it do two jobs at once.

Continuation. The window does not re-anchor. Re-anchoring is not merely redundant, it is wrong on the traffic this mode is for: a gapless stream has no 1.5-bit mark run to anchor on, so `ua_run`'s fallback takes the first candidate edge, which inside a byte is a data transition -- and the entire window after it decodes misaligned. Carried phase cannot do that.

De-duplication. Frames in the overlap were already emitted by the previous window and their positions are before `tmin0`, so the same test that excludes an in-frame edge excludes them. The test is `ei[k] >= tmin`, which **admits an edge exactly at `tmin0` and excludes everything earlier** — half-open, not closed. That is deliberate: the carried value is `t0 + (framebits - 0.5) * T`, and the half-bit exists so ordinary jitter cannot make a legitimate next start bit look as though it arrived inside the previous frame.

Excluding an edge at `tmin0` would put the slop off by one boundary and risk the first frame of every window. Stated as a closed interval this is the boundary a reader would correct the code to match, and the code is right. There is no separate dedup pass to get wrong, and no comparison of byte **values** -- which would have been the obvious approach and is unsound, because a stream of repeated bytes is common and two identical bytes at different positions are two bytes.

Asynchronous framing is what makes carrying phase safe: every frame still finds its own start edge, so `tmin0` is a floor on where the next one may begin, not an assumption about where it does. Drift between frames is absorbed exactly as it is within a window.

The seam advances only over frames actually **emitted**. Clipping at the byte cap and then carrying the unclipped window's end position would leave the seam past bytes the file never received -- invisible in a run that stops there, and a silent hole in one that resumes.

Constant stride

The window loop advances by a constant stride rather than resuming where the last frame ended. That costs ~1.6 % in redundant decoding -- the overlap, once per window -- and buys a window count known before the run starts, which is what makes the progress indicator honest.

Level measurement

Levels are an amplitude statistic; edges are a time statistic

Levels are an amplitude statistic; edges are a time statistic. That distinction is what makes a decode longer than memory possible without compromise.

`sig_levels` never looks at a sample's neighbours: it takes vmin/vmax, bins amplitudes into a histogram, walks the two densest humps and averages the samples inside them. Every one of those is order-free, so the levels can legitimately be measured on a **decimated** view -- every k-th sample, spanning the entire recording -- which costs one window of memory and gives exactly the population a monolithic decode would have seen. `sig_edges` cannot be treated that way, a threshold crossing being defined between adjacent samples, so edges stay window by window.

Averaging over the whole recording also absorbs slow drift better than any one window can: N full drift cycles are symmetric about the true levels and a quarter of one is not. So chunked mode measures the line **better** than frame mode rather than worse -- frame mode's capture *is* one window, so it has no whole recording to average over.

The opposite error is re-measuring per window, and it is equally real. A window that happens to be all idle -- entirely normal on a bursty line, and this mode exists for long recordings of bursty lines -- has only one dense amplitude. It would either refuse, or split that one hump and place the threshold inside a logic level, and the bytes on either side of that window would come from two different thresholds.

What remains unhandled is drift large enough that no single threshold fits the whole recording, past **~0.7 V of a 3.3 V swing**, where the monolithic decoder also fails. The fix there is a threshold that tracks slowly, and it is not built: choosing a tracking rate before seeing real drift on real hardware would be inventing a number.

Why ck_level_n is 20 000

How many samples the level measurement decimates the recording down to:

count	bins	modal	threshold	sig_levels
982	61	0.585	3.263 V	accept -- threshold ABOVE the high level
1963	122	0.325	1.651 V	accept
3925	245	0.176	1.647 V	accept
5233	327	0.235	3.295 V	accept -- wrong again
7850	490	0.137	1.648 V	refuse
10466	512	0.146	1.645 V	refuse
31397	512	0.145	1.645 V	refuse

`sig_levels`' bin count is floor(n/binocc) capped at nbins, so it saturates at n = 8192 and every count past that measures the same thing. Below saturation the coarse bins sometimes pick the wrong pair of humps outright, so a small count is not merely noisier. `sdec.n` is past saturation and is frame mode's own population, so chunked mode accepts and refuses a line where frame mode does.

Why the contiguous fallback is spaced across the recording

Decimation can alias past a sparse burst: a short message in a long recording -- the traffic the streaming modes exist for -- reduces to a handful of samples once the step is 140, and can reduce to none. Reading only the first window covers **20 000 of the 3 413 334** a full 32 kB recording holds, 0.6 % of the burst positions, hence `ck_probes` contiguous probes spaced evenly. The decimated view is still tried first, so the drift property that motivated decimation is untouched.

Priming and the pre-trigger reserve

An armed record opens on silence

`LoopUntilEvent` keeps a pre-trigger reserve ahead of the trigger, and on a device that had not started talking when Capture was pressed, every reading of that reserve is the line the arm was waiting on: **71 865 readings of a 32 kB recording's 1 437 294**, which is 3.6 windows of 20 000. So the first window can be entirely the wait.

Measured at 9600 Bd locked: a device starting at +3.0 s gave **0 bytes** and "no frame format fits the capture", where the same recording of one starting at +0.3 s gave **21 933**.

ck_prime_tries = floor(leadmax / W) + 2

Plus two, never fewer than two, and the 8 kB mode is why: its reserve is 2 035 readings short of a window, so its first window is 90 % the wait. The window straddling the end of the reserve holds a sliver of signal which need not frame, so the try after the reserve is the first certainly on the device's own traffic. **5 tries for the 32 kB mode, 2 for the 8 kB mode.**

It is a bound because every try costs a window. Over the reserve a try is a read, an edge pass and a format search with nothing to search -- 0.63 s at W = 20 000 -- and on a window full of frames that do not fit it is the whole 1.67 s. The worst case is 5 tries at 32 kB: **8.4 s to refuse a recording that takes 1.67 s to refuse on one try.**

Why sig_idle must exclude the leading run on an armed record

The priming window of an armed recording is the reserve, so its leading run is the wire before the device started, not the line's idle level. Left unset, `sig_idle` takes that run as the longest in the window and the polarity prior comes out of it -- and because priming's own decode then frames at that polarity, the format it returns is inverted and the whole recording inherits it.

Measured, an 8 kB recording armed on silence with the generator switched on at +3 s: leadrun 17 968 against a 17 965-sample reserve, run0 17 968, run1 232, idle 0, invert true, **2110 of 5484 bytes bad**. The byte count is right, which is why a count-only assertion passes on it.

Mode caps

ck_maxbytes = 32768

One number for every baud rate, so the UI states one expectation. 32 kB is 400-500 lines of 64-column debug text. 32 rather than 64 kB because the worst case then needs **3.41 M samples**, under half the reading pool, so the defbuffers at their 100 000 defaults leave room -- and where they do not, `sdec.bufmax_for()` shortens the window instead of refusing.

ck_keep = 8192

How many of a stream's last bytes are kept for the panel when a run ends. The run's product is a file -- 200 kB of decoded bytes does not fit in RAM -- so the tail is a fixed-size ring whose cost is the same whether the stream was 240 bytes or 240 kB.

8192 bytes is 35 pages of 240, and the whole of an 8 kB recording. At 720 (three pages) an 8 kB recording is pageable over its last 9 % and the rest is reachable only by unplugging the key.

Measured on the live app, since the argument rests on RAM:

figure	how it was taken
16 B per element	a Lua array of numbers: 8192 -> 128 KiB, 16384 -> 256 KiB
1887 kB heap	the app's live set, sampled straight after a `collectgarbage()`
3775 kB	`gcinfo()` 's threshold at the same moment
256 kB	allocated and freed with the event log clean

The count is a sawtooth between the live set and the threshold, hence the collect before sampling. Cost here is 128 KiB for the ring plus 128 KiB for `ck_tail_result()` 's copy.

The threshold is not an independent figure: 5.0.2 sets it to exactly twice the live count after a collect, measured at 1876/3752 loaded and 1887/3775 started. So it moves with the app, and crossing it triggers a collection rather than a failure -- which is why the argument rests on the 16 B per element and the 128 KiB, not on the headroom between those two figures.

Not 32768: that is 512 KiB twice over, never demonstrated allocatable. The file still holds all of it.

Window reuse between priming and decode

`ck_prime_step` 's read phase fills `win[1..cnt]` from position 1, and nothing between it and `ck_decode` writes into the table, so re-reading the first window costs W proxy round trips for values already in hand -- **0.435 s at W = 20 000**. The reuse is conditional on four things, because a format search that found nothing

in the reserve advances a window at a time and the table can hold samples from the middle of the record.

The four window-reuse conditions

`opts.primed` says a priming read happened; the four tests beside it say its contents still describe the window `ck_decode` is about to read. Each rules out one way the table can hold the wrong samples:

condition	what it rules out
`off == 0`	decode starting anywhere but sample 1
`opts.resume == nil`	a resumed slice, whose cursor is mid-record
`primed.W == W`	a window resized between priming and decode
`primed.at == 1`	a format search that advanced the primed read off position 1

`opts.primed` is cleared before the loop, so the saving is taken once and a later slice cannot inherit a stale table. Dropping any one test substitutes samples from the wrong part of the record for the first window's, which decodes as plausible frames at the wrong offsets rather than as an error.

The USB file API

No file call raises

Measured with the key present. Every failure posts an event instead:

call	returns	posts
`file.write` to a closed handle	returns	2200
`file.flush` on a closed handle	returns	2211
`file.close` on a closed handle	returns	2202
`file.read` on a closed handle	nil	2201
`file.read` on a WRITE handle	nil	2201
`file.open` on a bad path	nil	2216
`fs.is_file` bad path or garbage	false	nothing
`file.write(fh, {})` -- a table	returns	1138, 'parameter 2, expected type string'

The table row is the strongest: even a Lua argument-type error becomes a posted event. Across every shape measured -- dead handle, wrong mode, bad path, wrong argument type -- a `pcall` round a file call returned true, so a verdict taken from one is a constant. **Presence is the only file failure this application can observe.**

The rule is measured, not universal: these are argument shapes on a healthy key. A key that is present and REFUSING -- full, or write-protected -- has never been exercised. `vendor/repro-04b-full-key.tsp` would settle it.

A key pulled from under a running app

Key out, handle still open:

call	pcall	posts
`file.write(stale, s)`	ok = TRUE, err = nil	2200, 'File write error'
`file.flush` / `file.close`	ok = true	nothing
`file.open(path, ...)`	returns nil	2224, 'USB flash device not present'
`file.usbdriveexists()`	returns 0	nothing

`file.usbdriveexists()` costs **237 us** with a key in, names no path and posts nothing, so it is askable per line, per capture and per panel refresh.

The key-presence latch is not debounced

One absent `usbdriveexists` read latches, so a flapping key pays a full recovery per flap: the handle closes, the directory cache goes, keygen advances, the next write re-lists the 64 kB root and the byte log starts a new bytesNNN.txt. At 2 Hz that happens with nobody pressing anything. Two consecutive reads would avoid it and are deliberately not required: no spurious zero has been observed, and debouncing would delay the measured first-write-after-a-pull refusal to the second poll.

A key that was never there is not a key that went away. `ulog.on` and `ulog.dirok` are the marks a working key leaves, and the no-key branch still sets `keyout`, so a first insertion advances keygen even though nothing was lost.

file.mkdir

Returns nothing, never raises, and posts **2208** on failure -- including for a name that already exists, so a speculative mkdir files one event per call. A bare name always fails; `file.mkdir('/usb1/SERDEC')` succeeds silently. The reference's own `file.mkdir("TestData5")` example is wrong. Opening a missing path posts **2205**, so probing for the directory is not free either.

The only silent way to tell is to read `/usb1`, which opens for read and yields its raw FAT directory table. With no key even `/usb1` posts 2205, hence `usbdriveexists()` first.

Why the directory is called SERDEC

A name of eight characters or fewer in upper case is a single contiguous 8.3 entry, found by one `string.find`. FAT splits a long name across fields of 5, 6 and 2 characters with record metadata between them, so no search for the whole name matches:

```
SERIALFILES = false     no plain entry exists for an 11-character name
utf16_full  = false     the UTF-16LE long name is NOT contiguous
utf16_5     = true      only its first five characters are
SERIAL~1    = true      a tilde alias exists, but its number is not predictable
```

The root listing is **65536 bytes**, measured, which is why a confirmed directory is cached.

fs.is_file against the open probe

`file.open(path, MODE_READ)` uses the failure as the answer and posts 2205 at severity ERROR, the one severity `showevents` still raises a modal box for -- so finding a free name would produce one on every Save. `fs.is_file()` answers silently: `eventlog.getcount()` stays 0 across an existing and a missing path.

ulog.next_free's index and the wrapping scan

The index is a starting point, not an answer. Probing from zero makes repeated New Log presses post 2205s quadratically and raises a modal 'Multiple errors have occurred' box over the hex dump. The probe still runs forward from the index, because a file the index does not know about -- copied onto the key, or written by another prefix -- would otherwise be handed out and truncated by MODE_WRITE.

Exactly `limit` probes, wrapping, since the index may be stale high after deletions. Wrapping rather than clearing `idx_ram` and recursing: the retry re-reads the index file, which holds the same high number until a free name is found, so the scan repeats for ever. 5.0.2 has proper tail calls, so it never overflows the stack and a caller's pcall cannot break it.

On exhaustion the index file is left alone -- it costs one wasted probe per call, and that path must not need a working key. `idx_why` is set explicitly, because left alone it keeps the last allocation's sentence. The leaf only: `ui_notes` prefixes 'NOT LOGGING: ', and the full prefix measures 704 px against the 658 the note row shows.

LIN

The LIN checksum

Inverted eight-bit sum with the carry added back, modulo-255 rather than 256. `sum` stays <= 255 before every add, so one subtraction restores the invariant, and 255 - sum is the one's complement without a bitwise operator. Summing the data and the received checksum gives 255.

Diagnostic frames are classic by spec, so they are tried that way round; everything else tries enhanced first, LIN 2.x being the common case. Both sides of every comparison must exist: `nil == nil` is true in Lua and both sides can legitimately be nil -- `r.vals[]` for a lost byte, `li_csum` for a missing one -- so without the tests a response missing a data byte and its checksum validates out of two absences.

The style census excludes diagnostic frames: they are classic at any version, so counting them makes an ordinary LIN 2.x bus with a 0x3C frame report 'mixed csum'. Among the rest a mixture means a half-migrated stack, or a corrupted enhanced checksum landing on the classic value, which happens once in 256 frames. An untrusted frame does not vote.

The checksum's blind spot

Modulo-255 makes 0xFF congruent to 0x00, so swapping one for the other is invisible in every byte position: **0 of 256 detected across all 64 IDs and 4 positions.** With PID 0x11, data 0xFF and 0x00 both give checksum 0xEE. An agreeing checksum is therefore no proof that a byte the UART layer flagged arrived intact.

The checksum has **never been checked against a real frame** -- hand-computed cases only. `li_pid` is checked against the known headers: ID 0x3C -> PID 0x3C, 0x3D -> 0x7D, 0x00 -> 0x80, 0x01 -> 0xC1.

Break detection

`li_minbreak = 11` bit times. A break plus delimiter is at least 12 and a byte butted against the next is 10, so 11 separates them. Corroboration rather than an independent test: `li_break` also requires value 0x00 and a failed stop bit. What survives all three is a 0x00 whose stop bit flipped inside a stream with 2+ bit times of space, which splits one frame in two and fails both checksums.

Wiring

A 12 V single-wire bus against a 10 V best digitize range needs a 2:1 divider, 47 k + 47 k. Nothing below 20 k per leg, or a slave-only segment's pull-up reads recessive as dominant. A 24 V commercial-vehicle bus needs more than 2:1. `lin-divider-panel.png` is the drawing.

Distinguishing a divided bus from a transceiver TTL pin cannot work: 6 V against 5 V, and both decode anyway. So the panel note tests the recessive level only.

MIDI

MIDI truncation is reported, not dropped

A Note On that lost its velocity byte would otherwise simply not be in the list, so the message count would disagree with the byte dump and nothing would say why. Two ways it happens: the capture ended mid-message, or a fresh status byte arrived and abandoned it -- a byte lost on the wire, which is the common one.

UART framing

Why only 7 and 8 data bits are searched by default

The hostile sweep run both ways changes 5 of its 105 UART cases, and every one changes the same way. Its LIN cases cannot change, LIN pinning the width at 8.

case	7/8 only	any width
60 spikes 5 V 8 wide	8N1, 2 err	9N1, 1 err -- bytes all +256
1 V drift over 2 cycles	8N1, 3 err	9N1, 0 err
1 V drift over 12 cycles	8O1, 5 err	9N1, 1 err
noise+spikes+ringing+drift	8N1, 15 err	9N1, 9 err
genuine 9N1 stream	8N1, 5 err	9N1, 0 err -- the one real gain

Two mechanisms, the second being the dangerous one. The search keeps the highest-scoring candidate, so every extra format is another draw at winning by luck: 2 widths x 3 parities x 2 polarities is 12 candidates, five widths is 30. And **a wider frame launders the evidence of damage** -- 9N1 has an extra data cell and its stop bit lands one place later, so it absorbs whatever broke the 8N1 stop check into a data bit instead of failing on it. That is why the error count FALLS as the answer gets worse, and every byte is 256 too large.

6 data bits does the same thing one cell down: 6 bits plus a stop plus two idle bits is exactly a 10-cell 8N1 frame, so on any capture with a 2-bit gap both score 12 and the loser's bytes are silently wrong.

Why 9 data bits is never searched, even biased

Biasing is inert exactly when it is needed. `ua_margin`'s proportional term is 0.6 * sc and applies only when sc > 0; on a damaged line the honest reading's own score is driven to zero by its correct error count -- 12 frames with 3 bad scores 9 - 3*3 = 0 -- so the term is 0, multiplying by `rarewidth` leaves 0, and the margin collapses to its floor of 2. A 9N1 rival scoring 12 clears that trivially.

Measured on an 8N1 line with a 2-bit gap and stop-bit damage on interior frames, "any width" selected:

capture	result
12 bytes, 3 stop bits corrupted	9N1, nbad = 0, **9 of 12 bytes wrong**, every one +256
20 bytes, 6 stop bits corrupted	9N1, nbad = 0, **14 of 20 bytes wrong**, every one +256

ERR reads 0 -- the one number an operator uses to decide whether to trust the dump -- and those bytes reach the USB byte log and the saved report. `ua_refine_width` cannot walk it back: its collapse requires the top bit set in every error-free frame, and since 9N1 reports no errors the damaged frames are all examined and the first with bit 9 = 0 breaks the loop. The same damage that lets 9N1 win disables the defence against it.

1.5 stop bits

capture	result
5N1.5 at 9600, auto	reports 19200, exactly double -- 8 frames, 4 bad
5N1.5 at 9600, baud forced to 9600	all six bytes exact, 0 framing errors

The frame layer handles it; `sig_bittime` does not. A 1.5-bit stop makes some pulse widths odd multiples of HALF a bit, so the greatest common bit period collapses to half the true one -- the "errs short" failure, but structural to the format rather than caused by a glitch, so noise tolerance cannot help.

Left unfixed deliberately: widening `sig_bittime` to resist a structural half-bit would put every common format at risk to rescue a doubly-rare one. 1.5 stop bits belongs to Baudot, which needs 5 data bits too, already outside the default search. The remedy is the Baud Rate field, and the failure is flagged rather than silent.

Baudot used a 7.5-unit character -- 1 start, 5 data, 1.5 stop -- and at a 22 ms bit that puts 45.45 Bd at the standard 60 words per minute. The mechanism needed settling time that was not an integer number of bit periods.

ratemargin = 0.04

Just inside the measured **5.26 %** cliff: 0.5/9.5 for 8N1, half a cell accumulated over the 9.5 bit times from the start edge to the stop sample. Advisory by design, warning at the point where the next percent could start corrupting bytes silently rather than waiting for corruption that by construction leaves no other trace.

qgate = 0.8, and why pulse-width consistency is needed at all

On a badly corrupted 9600 baud capture, 7N1 at 7200 baud frames **54 of 54 bytes with zero errors** while the truth frames 40 with 15 errors. A 9-cell frame at 13.87 samples spans 124.8 samples and a 10-cell frame plus a 2-bit gap at 10.417 spans 125, so the alias re-samples every frame at the wrong offsets and still lands its stop bit in the gap every time. No frame-error score can prefer the truth. Pulse-width consistency is decisive -- **0.855 against 0.497** -- because the real widths are multiples of the real bit time and of nothing else.

Why sdec.ratios is asymmetric

Integers to 10, reciprocals only to 1/6. `sig_bittime` fits the greatest common bit period of the observed pulses, so it errs SHORT: one narrow glitch pulls the fit to a sub-multiple, hence integer ratios well above 1. Erring LONG by a factor k needs every pulse in the capture to be an exact multiple of k bit times, and inside a 10-bit frame that stops being constructible past k = 6, where it would mean no run shorter than six bits.

ua_popcount's lookup table

The repeated-halving form is **11 % of every C call the app makes**, measured over 300 bench cells, because each iteration costs a `math.mod` and a `math.floor`. It is called once per CANDIDATE frame during the format and parity search, not once per decoded byte, so it runs thousands of times per capture. A 512-entry table over the 9-bit domain is ~8 kB against the ~500 kB a 32 k-sample capture already holds, and is built on first use so an app that never decodes a parity format never pays.

The loop is kept for an integer wider than the table, fused to one C call per iteration: `v * 0.5` has a fractional part exactly when v is odd, the same test as `mod(v, 2) == 1` for one floor instead of a floor and a mod. Not a general fallback -- a negative returns 0 and a non-integer counts fractional halvings -- but in-tree callers only ever pass 0..511.

Three-tap majority sampling

A single sample is one ADC conversion, so a noise excursion at that instant flips the bit; three taps spanning 40 % of the cell need two wrong together. Skipped below 6 samples/bit, where the outer taps would sit too close to the cell edges.

Two taps that agree settle a majority of three, so the third is not read: with the centre and lower tap on the same side of the threshold, `ones * 2 > votes` yields the same bit whether the upper tap is counted, counted the other way, or out of range -- all six cases check out. On a line clean enough to decode the pair agrees on essentially every slot, removing one `math.floor`, one bounds test and one array read per bit.

`ua_frame` repeats `ua_bit`'s sampling inline, expression for expression including the order of the floating-point operations, because `floor(x + 0.5)` at a cell boundary is decided by the last bit of x. A function call with its global look-ups costs about as much as the sampling it wraps.

Why the anchor search is bounded by `n`

The edge arrays span the whole capture and `n` need not: `ua_probe` scores a format over `ua_probe_n` samples from the first edge, and the edges past that are still in `ei`. Unbounded, the search returns an anchor beyond `n`, whose first frame reads off the end, comes back 'truncated' and breaks the walk before one frame is emitted -- so the probe reports that nothing frames while the format is right.

The failure needs the anchor reachable in the capture and not in the window, which a block payload produces exactly: 64 or 128 bytes of 0x55 put a one-bit mark run between every frame and never satisfy the 1.5-bit test, while the 0xAA block after them ends every frame with mark, mark and does. The anchor lands in the 0xAA block, 129 bytes past a 48-byte probe window. Measured over the soak plan: **106 of 13 416 points**, every one v90 or v94 and no other vector, going to zero at every cap from 2000 samples up.

The bound is `n`, not "room for a whole frame". Requiring room for a frame also discards an in-window anchor that cannot complete one, and the fallback then frames from the window's start -- mid-byte on a payload with no 1.5-bit mark run of its own, so every frame carries a valid start and stop bit and a shifted value. Measured on 200 bytes of 0x12 with one 0xAA near the end: **201 frames, 201 marked good, every byte 0x22.**

A capture that began mid-byte

Measured on the instrument, 8E1 at 9600 into the front terminals: the frames ahead of the first idle come out at pitches of **11, 12 and 14.9 bit times instead of 13**, and every frame after it is exact. The cause is the anchor -- for the first edge the mark run is bounded only below, by the start of the capture, so two bit times of apparent mark is indistinguishable from a run of 1s mid-byte. An armed capture legitimately starts a few samples before a real start bit, so requiring a longer run cannot resolve it.

headsusp is a region, not a count

`ua_run` sets `r.headsusp` when the frames before the first idle gap carry errors and the frames after it do not -- a capture that began part way through a byte. It is the count of bytes before that first inter-message **gap**, so it is a conservative region and not a count of bad bytes. The panel reported it as one: **"the first 79 bytes are misaligned" over a capture with 3 flagged bytes.**

Two figures fix that. The **last** flagged byte in the region makes "the first N" literally true, and the number flagged lets `ui_notes` swap count for extent: `nbad - headnbad + headbad`.

Both are computed in `ck_decode`, because only there is the head. A recording leaves its retained **tail** on screen, and a 32 kB tail starts at byte **24577** -- its `errs` array does not contain the opening bytes at all, so `ui_notes` cannot count them however it is written. A run keeps no `errs` of its own either, only per window, so ERR cannot recount the head later.

Only the window that produced the **first bytes** can have a misaligned head, and `tot.nf == 0` tested *inside* the emit, before the increment, is what identifies it. Not "the first window": with a lead-in the first window can decode nothing, so the same test outside that block stays true into window 2, and a later window's head is a seam the overlap machinery already handles -- claiming a misaligned start there is worse than saying nothing.

ua_edge_frames = 3

A capture of a continuously busy line starts and ends wherever the trigger and the buffer end put it, so the first and last frames are routinely sliced through the middle, and a gapless stream has no idle to anchor on so it also resynchronises a few frames in. Measured: without the exclusion, **every 9600-baud capture of a healthy line reports ERR 1 or 2** with the errors in the first row, colouring it red on a signal with nothing wrong. An error count never zero on good data is decoration, not a fault indicator.

ua_margin: why a plain '>' is wrong

Several formats are genuinely indistinguishable on the wire, and the shorter frame of such a pair always finds more frames in the same capture, so it always scores higher. 0x55 at 8N1 with a two-bit gap has a 12-bit period, and three 8-bit 6N1 frames tile two real frames exactly, so **6N1 decodes 9 error-free frames where 8N1 decodes 6**. Both are complete, consistent descriptions; no signal processing separates them.

The rule is therefore proportional -- a rival must win by >60 % -- which turns the search order into a commonality prior. Being scale-free it behaves the same on a 6-byte capture and a 600-byte one, and real evidence clears it easily: **a genuine 5N1 scores 8 against 8N1's 1.**

rarewidth = 8

At 8 a rare width must score nearly six times the incumbent and every genuine case still wins. **10 is the largest value that keeps every genuine 5N1 and 6N1 case decoding, and 12 breaks the tightest** -- 8 bytes of back-to-back 5N1, where 8N1 accidentally frames 4 of them and scores 1, so the rival needs to clear 0.6 * k. 8 leaves headroom rather than sitting on the wall; the 5N1/6N1-at-every-gap tests are the guard.

The bias multiplies the proportional term only, never the absolute floor. The floor stops noise deciding on a handful of frames, and quadrupling it made a rare width unwinnable on exactly the short captures where it is easiest to be sure: genuine back-to-back 5N1 over 8 bytes scores 8 against 8N1's 1, and a floor of 8 rejected it by one point.

polmargin = 4, and the inverted-ASCII trap

Three consecutive captures of the same 9600-baud line at 1 MS/s: the first decodes correctly, **the second flips to inverted and returns 18 frames with ZERO errors**, the third inherits it. At 104 samples/bit a 19 000-sample window holds only ~18 frames, and on that little evidence the inverted reading of ASCII text can out-score the correct one -- ASCII's always-zero top bit manufactures a periodic fake start edge nine bits after every real one. Self-consistent, confident, every byte wrong.

`sig_idle` sets `idle_weak` when the capture contained no idle, in which case the polarity prior is the signalling standard rather than an observation -- a single-supply line marks high, a ground-straddling one is RS-232 and marks low. A frame-count contest on one window should not overturn that on a 60 % margin, so the rival needs ~3.4x. 4 is the smallest value that blocks the 18-frame win while leaving every offline inverted-line test decoding; those captures contain idle, so the multiplier does not apply to them.

minsabit, and why the floor exists at all

Below the 4 samples/bit floor a clean synthetic waveform still decodes -- the harness recovers **13 of 13 bytes at 2.5 samples/bit** -- which is why the gate is explicit rather than left to emerge. Nothing real is that clean, and a decode that only works on a noiseless line is a trap.

plaustol = 0.02, on both ends

A genuine 250 kBd line measures **3.99998 samples per bit and 250 018 baud**, so refusing it for being two hundred-thousandths under its own floor would be a bug. Both ends carry it because a floor without it refuses the mirror case: a 50 Bd line whose fit measures 100.05 samples instead of 100.00 computes to **49.975 Bd and is refused**, while 51 Bd with identical jitter decodes byte-exact.

The median-pulse ceiling

Every pulse is a whole number of bit times, so every observed width is at least one bit and the median is too: a candidate longer than the median claims most pulses are shorter than a single bit. This rules out the long rescalings a damaged capture rewards -- at 1 V of drift the fit is right (**10.36 samples against a true 10.42**) but frames badly, and an 8x rescaling framing one byte takes over and reports 1200 baud.

ua_submultiple: the odd-multiple test

Halving is the dangerous direction: every pulse that is a multiple of T is also a multiple of T/2, so the q gate cannot separate them (**measured 0.99999999 for both**) while the halved reading finds twice the frames.

Odd multiples separate them. At the true bit time some pulses are an odd number of cells; at half it every pulse is even. Not the median -- all-0x00 is a nine-bit low run, so a median test rejected the truth. Even divisors only, and that is structural: an even divisor multiplies every k by an even number so nothing odd survives, while an odd one leaves k's parity as it was. Measured on the fox at 31250 Bd, 1 MS/s, odd/seen for 1/2..1/6: **0/358, 248/344, 0/333, 220/305, 0/305**. So 1/3 and 1/5 always pass whatever the payload, which is why `ua_minrun_absurd` exists.

It is a prior on real traffic, not a structural proof. A payload can have no odd run at all: repeated 0xE6 at 8N1 with a 2-bit gap gives runs of 2,2,2,6 -- **0 of 159 odd**. Measured on that case and on 0x33/0xCC/0x0F/0x66/0x00 at three gaps, the gate changes no answer for the worse and fixes two (0xE6 gap 1 refused -> correct, 0xCC gap 1 19200 -> 9600).

Both constants measured over **368 640 paired decodes**. A fraction rather than zero: jitter leaves 6-13 stray odd multiples per ~1000 pulses, and demanding exactly zero repairs only 398 of 708 cases. Odd fraction is 0.011-0.013 at a halved T against 0.587-0.604 at the true one.

Only bit runs may vouch, bounded by `maxmult`: a multiple beyond it is inter-byte idle carrying no timing information. A 42-bit loop-seam gap gives **k = 169**, whose parity is set by the capture phase rather than the format, and one such width was enough to silence the gate and let ratio 0.25 through.

submult_maxrunlo = 2.5

Covers the odd divisors `ua_submultiple` structurally cannot. A ten-bit frame opens with a one-bit start bit, so hundreds of pulses hold single-bit runs: **1.00 bit times at the truth against 5.00 at the 1/5 candidate** that displaced it.

The bound must sit below 3, because a 3x misfit puts the shortest run at exactly 3.00 candidate bit times and 3x is the largest family on the bench. Measured on the LIN-break vector at 19200/57600/76800, 60 capture openings each: **2.0, 2.5 and 2.9 all report the true rate every time; 3.0 and 3.5 hand back 42, 60 and 60 wrong.** 2.5 is the middle of the working range.

What it costs: a payload whose shortest genuine run is 3 bit times or more cannot have a 1/3 or coarser rescaling admitted once the fit snaps. Repeated 0x00 at 8N1 with a two-bit gap is that shape, and reports a third of its true rate with this test and without it alike -- nothing lost there, but the exposure is real.

Over **1677 plan cells x 200 presses, 2.5 moves standard-rate misreports from 2604 to 882** and makes no cell worse. The 5th percentile rather than the minimum, because one 2-sample notch would silence this as one loop-seam width silenced the odd test -- and that notch is itself how a fifth-of-the-bit fit arises.

ua_fit_corrupt: med/Tfit separates two faults

The one number that separates "the line is faster than this instrument" from "impulse noise captured the fit", and structural rather than tuned: in async serial the median pulse is two bit times, because a ten-bit frame plus a couple of idle bits produces runs whose middle value is about two.

Measured med/Tfit is **2.0 for every healthy capture tried** -- 9600 clean, 250 kBd, 285 kBd, 921 kBd, and all the drift cases -- and **3.0, 11 and 13 for the three spike-corrupted fits**.

So when the fit is implausible AND consistent with its own widths, the line really is out of range and rescaling would manufacture a plausible-looking wrong answer. When it is grossly inconsistent, the fit is what is wrong and rescaling is right.

Why ua_cands adds median-derived candidates

The least-squares fit has one failure mode the ratio ladder cannot cover. **Forty 2-sample spikes on a 9600 baud line** produce forty narrow widths that corroborate each other, so seed corroboration passes them and the fit lands at **0.95 samples against a true bit time of 10.42** -- a ratio of 11, which is not on the ladder and never will be for every such case. Coverage scoring cannot reject it either, because a shorter trial period reclassifies the contradicting long widths as idle and drops them from its own denominator.

The median cannot be moved that way: spikes are a minority of the widths however many there are. Real widths are 1..maxmult bit times, so med/k for small k brackets the truth from a direction impulse noise cannot corrupt, and **med/1 alone recovers all three spike cases the ladder missed**.

Why the rescaling search is gated

Halving the bit time roughly doubles the frames found, so any score counting frames, bits or coverage is biased toward a smaller bit time: **on a repeated 0x55 square wave the true bit time frames 6 bytes and a quarter of it frames 30, all error-free.** Every rescaling of an alternating pattern is self-consistent, so no scoring function can prefer the truth.

`sig_bittime`'s two blind spots on synthesised captures: the widths can share a common factor (0x00 at 8N1 with a two-bit gap yields only 9- and 3-bit runs, best-fitting **three** bit times), and a sub-bit glitch can score higher than the truth because a shorter trial period draws fewer long widths into its coverage measure (a 2-sample notch produced a fit at **one fifth** of the real bit time, **q = 0.984 against the truth's 0.962**).

So the width fit is trusted whenever it snaps to a standard rate and frames something; otherwise the snapped rescalings are searched, real devices using standard rates being the strongest prior available.

The same waveform at two rates

A common factor in the pulse widths does not reliably produce a non-standard rate, because half of a standard rate is usually also standard. **Eight 0x00 frames at 9600 7N1 with a one-bit gap fit 4800 exactly, and read as four 0x08 bytes at 4800 8N1 with zero errors** -- and that is not an error, because the two encodings are bit-for-bit the same waveform, verified sample by sample. Which device is on the wire cannot be recovered from it.

Both readings are legitimate and the commonality prior even favours the 8N1 one, so the operator is told: a rescaling that also snaps and explains materially more of the capture is reported in `baud_note`. Perfectly periodic traffic creates this; irregular inter-byte gaps break the coincidence, so it is largely a synthetic-stream phenomenon.

ua_probe_n = 4000

The search is the app's worst latency offender: **4.376 s to auto-detect 38400 baud** on the instrument, against a 500 ms press-latency budget. The cost is multiplicative -- 3 widths x 3 parities x 2 polarities is eighteen full framing passes per candidate bit time, and `ratios` offers thirteen candidates before the integer sub-multiples -- so a probe pass runs upward of two hundred times.

A probe's output is a relative score used only to rank candidates, and ranking is settled long before the last byte. The final decode is unaffected; only the ranking is truncated.

4000 is measured, not chosen. Sweeping the cap against the 148-case hostile suite, edge-anchored:

cap	result
none, 16000, 12000, 8000, 6000, 4000, 2000, 1500	127 ok, 21 degraded, 0 WRONG, 0 raised
1000	126 ok, 22 degraded, 0 WRONG, 0 raised
500	125 ok, 22 degraded, **1 WRONG**, 0 raised

Discrimination is intact down to 1500, starts costing at 1000, and produces a wrong answer at 500. 4000 keeps 2.7x margin over the degradation floor while cutting the search's work 5x against a 20 000-sample capture. On the host the whole search costs **3.3x more uncapped than at 4000**, which puts the instrument's 4.376 s at about 1.1 s. It is ~48 bytes at 8.33 samples/bit.

Re-run that sweep (`SDEC_PROBE_N=N lua tools/stress_serial.lua`) before changing it. The failure mode of too small a value is a quietly worse answer, not a crash, and only the suite makes it visible.

The cap does not decide whether a block payload decodes, though it can look as though it does. v90 and v94 lost 106 of 13 416 soak-plan points to 'no frame fits' and the count fell monotonically as the cap rose -- that was the frame anchor being chosen outside the window the probe scores, not the window being too small. See 'Why the anchor search is bounded by `n`'.

ua_best's split

`ua_best_probe` scores one candidate and is the expensive unit, **~76 ms measured at 38400 baud**. A candidate probe is the loop's atom -- 18 framing passes with no state carried between candidates except the best-so-far -- so the checkpoint is four values and a slice boundary cannot land anywhere awkward.

Only defend a fit that landed on a standard rate

The sub-multiple gates in `ua_best_probe` exist to stop a good fit being displaced by its own halving. When the fit does not snap it is not a good fit, and a rescaling is the only route to the truth, so `snap1` is what makes the pair of gates safe.

Repeated 0xE0 at 8N1 with a two-bit gap is all 6-bit runs, so the fit lands on **1600 Bd** (not snapped) and the truth is the ratio-1/6 candidate. Its multiples are all k = 6 -- even, so an ungated odd-multiple test rejected it and handed 4800 to the ratio-1/3 candidate: **20 frames of 0x1C with zero errors against 40 exact 0xE0.** Behind the `snap1` gate neither sub-multiple test is consulted on that capture and the 1/6 truth stays reachable.

Only displace a fit that is failing

A margin alone cannot decide it. On an alternating pattern every rescaling is error-free, so 0x55 frames **6 bytes at the true rate and 30 at a quarter of it**. What separates the cases is whether the FIT explains the capture: measured, a 9600 line fitted 19200 with 35 bad frames (**score 11**) against a snapped 9600 at **~150**. A clean fit is never displaced.

A negative-scoring rescaling may not displace a snapped fit

`ua_score` is ngood - 3*nbad, so a score at or below zero means the errors outnumber the good frames threefold: the candidate does not describe the capture, and ranking two such candidates against each other is ranking noise.

Measured on a waveform where every candidate fails -- a LIN bus with no break delimiter, where the merged break and start bit destroy the framing. The true 19200 scores **-20** and its own half scores **-15**, so the half wins on -15 > -20 and is reported as a confidently snapped 38400. The fit is right, snapped, and has the best pulse-width q of any candidate; nothing about -15 is evidence.

Both halves of the guard are load-bearing. `sc1 ~= nil` means the fit passed its own plausibility and q admission tests, so there is something worth protecting; when the fit is inadmissible a negative-scoring rescaling really is the best answer available and must still win -- on noise plus spikes plus ringing plus drift at 9600 the fit lands at **3.47 samples/bit**, and the only rescaling to survive the q gate is the truth at 9600 scoring **-5**. And `snap1` matters because a standard rate is the strongest prior there is: when the fit does not snap and a rival does, that is real evidence and the rival should win however badly both frame.

The text tie-break

The guard above defends a fit that snapped to a standard rate. A fit that snapped to nothing gets no margin at all: it falls through to a bare `altsc > sc1` and is displaced by **one point**. Measured on a 3572 baud capture of 512 bytes, the width fit scores **47** and its own double scores **48**, so half the bit time wins by one frame in several hundred and the rate is reported as 7200 -- 3572 x 2 is 7144, which is 0.78 % from 7200 and inside `snaptol`, so the snap launders the doubled fit into a standard rate.

One point over hundreds of frames is a coin toss, and `sdec.ua_tie_margin` is the width of the band called a tie. Inside it the decoded bytes decide: prefer the bit time yielding more of A-Z, a-z and 0-9, counted over good frames only across the whole capture. A tie on text changes nothing.

It is inert on binary payloads, both counts being zero, and that is what makes it safe. The case it must not disturb is a capture whose pulse widths share a common factor -- all-0x00 at 8N1, where the fit sits on a multiple of the bit time and the rescaling is the only thing that recovers the truth. Those displacements are won on text **0 against 0** at a one-point margin, so they fall through untouched. At a margin of two they do not: there the fit reads **4 letters against the rescaling's 0**, which is why the band is one point and not two.

The count must be taken over the whole capture, not over `ua_probe_n`. The probe reads 4000 samples, about **47 bytes** at 8 samples a bit, and a payload whose head is padding shows no text at all in that window -- the 3572 baud case above reads **0 letters** in the first 47 bytes and **128** across the capture. `ua_probe` reports which format it ranked best so the tie-break can re-run that one alone, two decodes rather than the 18-way search again, and only at a tie.

Measured over the soak plan, 8 capture phases a cell, 13 416 points: **rate 107 to 100**, failing points 110 to 103, cells failing at every phase 10 to 8, and `bleed` unchanged at 1050. Over five laps and 67 080 points, **335 to 327** with every other counter identical. The unit suite goes from 1481 to **1492**, 0 failed.

What makes a text count evidence

A bare count decides nothing, and "inert on binary payloads because both counts are zero" is false. A wrong bit time re-slices the stream and **fabricates** letters where the truth has none: a 9600 baud line carrying the byte ramp 0x01..0x1F decodes **0 letters at the truth and 7 at twice the bit time**, so a count alone keeps the wrong bit time and prints the truth's own 0 as the loser. All-0x00 is the one binary payload that is safe by accident, both counts being zero, and it is not representative.

Two floors, and a count must clear both. `sdec.ua_textmin` is 16 letters absolute: instrumented over the whole plan, every firing reads **52 letters or more** -- 52, 61, 64, 64, 116, 125, 128 -- while the fabricated counterexamples read 4 and 7, so any floor between 8 and 52 separates them with no measured gain lost. `sdec.ua_textfrac` is 0.25 of the good frames: **62 of 256 byte values are alphanumeric, so 24.2 % is the density chance alone reaches**, and the fabricated case is 7 letters in 60 frames, 12 %. Real text runs well above half.

Both floors together cost nothing: the plan verdict is **bad 103, rate 100, bleed 1050**, identical to the unfloored tie-break, and the 0x01..0x1F ramp keeps the truth.

The tie-break is also the second unbounded step in `ua_best_end`, two full-length decodes, so it ticks. `ua_best_probe` gives one tick per candidate for the same reason.

Collapsing an always-one top data bit

A data bit that never changes carries no information, and in the top position it is indistinguishable from the stop bit that has to follow it: N data bits whose MSB is always 1 describe exactly the same cells as N-1 data bits with the stop bit one place earlier. Both readings are complete and consistent, so error scoring cannot separate them, and the longer frame wins whenever noise gives it a single extra frame -- which is how a spiked 9600 baud line comes back as 9N1 with **every byte 256 too large**.

The shorter reading is also almost always the physically right one. Nine-bit formats exist for multidrop address marking, where the ninth bit varies by design; a ninth bit stuck at 1 is not a nine-bit device. Only an always-one bit qualifies -- an always-zero MSB is ordinary 7-bit ASCII in an 8-bit frame -- and 7 data bits plus MARK parity genuinely does put a constant 1 there, so the collapse is reported rather than silent.

The test masks off the bits already dropped. Testing the raw value instead works only when every strip happens to be valid: a 6N1 stream read as 7N1 has values d + 64, and once bit 6 is dropped the test for bit 5 must look at d, not at d + 64 -- which is always >= 32 and so always "another constant 1". It stripped

twice, the 5-bit re-decode then framed worse than the 7-bit one, the ngood guard threw the whole collapse away, and **6N1 traffic came back as 7N1 with every byte 64 too large and zero framing errors**.

Re-decode rather than shifting the values in place: the shorter frame has a shorter framebits, so its resync point sits earlier and it can pick up a start edge the longer reading steps over. Shifting values keeps the longer reading's frame count while claiming the shorter format.

Separating 8N1 from 7E1

The one pair error scoring cannot tell apart, since 7 data bits plus parity occupy the same ten cells as 8 data bits with the stop bit in the same place. 7E1 traffic decodes as 8N1 with zero framing errors and wins on search order. What gives it away: under the 8N1 reading bit 7 would equal the parity of bits 0..6 in every frame, a 1-in-2 coincidence per frame.

A repeated byte is genuinely ambiguous -- 0x55 as 8N1 and as 7E1 give identical waveforms -- so the vote needs >= 3 distinct values and the common format wins without them.

The parity vote's two tiers

The sample size a verdict needs depends on whether it tolerates dissent.

agreement	gate
unanimous	`par_minvote_all` = 6
any dissent	`par_minfrac` = 0.95 **and** `par_minvote_frac` = 20

The unanimous bar must stay small: 'Hello, World!' is thirteen bytes and is legitimately 7E1, so a bar of twenty would refuse to classify a whole class of real short captures. Raising it broke six tests. Tolerating a dissenter is the riskier claim, so it earns the larger sample. Against the null hypothesis of random 8-bit data with bit 7 matching parity at p = 0.5, 95 % of 20 votes is **2e-5** and of 140 is **1.4e-31**, both doubled for the two parities tried. A dissent can never buy a promotion that unanimity could not.

par_minboth: both bit-7 states, or it is not evidence of parity

`ndist` alone does not protect. **48 of the 95 printable ASCII characters have even popcount**, so an 8N1 payload drawn only from those has bit 7 always 0 and even parity 0 in every frame -- perfect apparent 7E1 agreement over arbitrarily many distinct values. A real parity stream exercises both states: of the 94 visible glyphs, **46 set bit 7 under even parity**. A constant-parity payload genuinely lacks the evidence to prefer 7E1 over 8N1 and must stay 8N1.

Why the parity vote skips the capture boundaries

Always, not only when `headsusp` says so. Unanimity will not do and `headsusp` cannot rescue it: `headsusp` needs a framing error in the head, and a misaligned head that happens to frame cleanly as 8N1 produces none. Measured on hardware with v78 (7E1, gap = 0): **4 of 8 captures come back with zero flagged frames and every byte carrying a spurious bit 7** -- parity absorbed into data bit 7, nothing flagged, because 8N1 is a genuinely self-consistent reading of a 7E1 waveform. **47 of 48 bytes have bit 7 equal to the parity of their low seven**, the exception being the first frame. With gap = 0 there is often no `idle1` either, so `headsusp` has two independent reasons not to fire.

Three frames at the head, not one: a gapless stream has no idle to anchor on and resynchronises a few frames in (see 'ua_edge_frames = 3'). One at the tail, which the buffer end slices.

`headsusp` is the region before the first gap, not a count of damage, so on a gapless stream there is no gap to end it and it reaches almost the whole capture: measured on v78 at 348 and 630 Bd, it claims **130 of 136 and 120 of 122 frames while ngood is 135 and 121** -- one frame in error. Excluding all of that leaves nvote 5 and 1, which fails `par_minvote_all` and `ndist`, so the promotion never runs and a 7E1 waveform is reported as 8N1 with every byte carrying a spurious bit 7.

A gapless stream has one alignment for the whole run, so if the frames decode without error the alignment is self-consistent and the only open question is 8N1 versus 7E1 -- which is what the vote settles. So `headsusp` is honoured only while it leaves a usable vote; where it would starve one it carries no information and the measured damage bounds the head instead.

A dissenting voter forces a re-decode

Rewriting every value to mod(v,128) is sound only when every voting frame agreed: then no frame is being relabelled 7E1 while actually failing that parity. Under a fraction the dissenters would keep a clean bill of health while contradicting the format just assigned -- the fix would itself become a silent wrong answer, which is the defect class this decoder exists to avoid. So a dissent forces a real re-decode, where a dissenter becomes a real parity error.

The re-decode widens `headsusp` and never overwrites it. The re-decode finds its own suspect-head extent and the 8N1 result's is nil whenever that pass framed the head cleanly, so overwriting loses the evidence and ERR then counts no head on a capture this format says is misaligned. Measured over the firings on the 7-bit random payload: **2 in 7 differ, at 14 and 57 frames, narrowing to 5 and 4**. Both formats frame 10 bits, so the figures share a unit; the wider is safe because `headsusp` only bounds `ua_head_bad`'s scan and the narrowing is what prevents over-claiming.

The format has to be stamped on by hand. `ua_run` returns values and errors, not which format produced them -- the search loop does that when it picks a winner -- so the re-decode came back with nbits/par/nstop/invert all nil and only score and headsusp copied: the branch whose purpose is to change the format returned one with none. It surfaced as a crash in `ua_note_fmt`'s %d, so `capture()` raised while describing a decode that had succeeded and the bench filed 'no bytes decoded'. **14 of 256 offsets offline**, identical at c7ca3e9. The strict path escaped it by mutating r in place.

ua_alts must run before ua_refine_parity

The exclusion test compares each candidate against the format that won the search, and `ua_refine_parity` rewrites r.nbits and r.par in place, so running afterwards compared 8N1-scored alternatives against 7E1. `ua_refine_parity` also sets `fmt_note`, which the old version bailed on, so whenever the 7E1 reclassification fired **no alternative was ever reported at all**. Composing the note is a separate step (`ua_note_fmt`), which is what lets both notes coexist.

The retracted bit time is kept for autoset

A failure past the bit-time stage retracts `sdec.baud`, or the panel shows a confident '216018?' against BYTES '--': a number that looks measured when nothing was decoded. A bit time that frames nothing is not a baud rate, it is an artefact of noise.

The retracted value is kept separately in `baud_probe` for the one caller that can act on it. It must not be displayed, but `autoset()`'s first pass runs at 1 MS/s, where a 20 000-sample window is 20 ms and holds only six bit times at 300 baud. The bit time is measurable there and a frame is not, so 'no frame fits' from the probe is an instruction to re-capture slower rather than a dead line. Discarding the measurement made **every rate below ~1200 baud fail outright with the answer already in hand**.

A forced rate the device is not using

The one dangerous failure this decoder has, so a forced rate is cross-checked against the wire. Measured, 8N1 forced to 9600, 8 seeds per point:

offset	result
-4.5 % .. +5.0 %	byte-exact, 8/8
+6 % and above	SILENTLY wrong, 8/8 -- zero framing errors, corrupted bytes
-5 % and below	flagged wrong, 8/8 -- framing errors raised

The cliff is 0.5/9.5 = **5.26 %**, exactly the textbook figure: the stop bit is sampled 9.5 bit times after the start edge, so accumulated error must stay inside half a cell. The asymmetry is what makes this worth a guard. Too fast and the sampling point drifts early, staying inside the previous cell, so every frame remains internally self-consistent -- nothing downstream can detect it and the panel shows confident garbage. Too slow and it drifts past the stop bit into the next start bit, which fails the stop check and is already reported.

So the fit is measured even though it is not used. It costs one `sig_bittime()` over the width array -- no per-sample work, the widths are already built -- and it is the only evidence available that the forced number is wrong. Advisory, never fatal: the operator typed that rate deliberately and may be right when the fit is not, a 5N1.5 Baudot line defeating `sig_bittime` structurally, which is exactly when someone types the rate in.

sig_bittime() is not pure, so the advisory call is fenced

Its first act is to nil and re-publish `sdec.bittime`, `baud`, `baud_raw`, `fitq`, `snapped` and `snapfirm`. Calling it as a cross-check therefore overwrites the forced state with a measured one and nothing downstream knows: `chunk_decode` reads `sdec.bittime` to build fmt.T, so a streaming run decodes the whole recording at the measured rate instead of the locked one and writes that to the USB byte log; on the frame path the panel draws the measured BAUD in the locked green whose entire justification is that a forced value must not look measured. Leaving `snapfirm` out of the snapshot lets an advisory measurement add or remove a forced rate's '?'.

So: snapshot, call, restore unconditionally. A local function rather than an inline block, and not a pcall: the body has two early returns -- 'no measurement' and 'an integer rescaling, stated' -- and inlining them would return out of `decode_from()` and skip the restore. Nothing in it reaches the instrument, and `sig_bittime()` is called bare in the unforced branch of the same function, so a guard here would be a guard around our own arithmetic.

An integer rescaling is ambiguous, not innocent

`sig_bittime` fits the greatest common period, so a ratio near 1/2 is usually the fit erring short on a healthy capture -- but it is exactly what a device running at twice the forced rate looks like, and that decodes with no framing error and wrong bytes. The two cannot be told apart from the ratio, so the measurement is stated rather than the disagreement suppressed. Raised by review with a concrete repro: **force 9600, transmit 0x43 at 19200, get 0x78 with zero errors.**

The note row clips, so the verdict leads

`ui_note_px` is 700 and the cell clips with an ellipsis, so a note that builds up to 'WRONG' loses the only word that matters. Measured at **990 px**, the operator read '...the bytes are...' and nothing after it; the rate-disagreement note clipped at **914 px** and read '-- bytes may be...' with WRONG cut off. Both are front-

loaded and trimmed to **699 px** at the widest field values they can print (8x, 250000 against 104857). The percentage loses its decimal to buy that room -- +6 % against +6.0 % changes no decision -- and 'you set' stays because whose number it is is the actionable part.

relock_badfrac = 0.25

Doubling the generator's rate under a 19200 lock made every frame fail, and repeated Capture presses kept showing all-red: the app had no way back, and the operator had to type 0 into Options to re-enable detection. A rate that explains nothing is not worth keeping.

0.25 is measured. 'Every byte is in error' is what the panel looks like -- a row is painted red if any of its 16 bytes failed, so a third of frames bad reddens essentially every row -- but the actual interior-bad fraction for the bench case (38400 on the wire under a 19200 lock at 8.3 samples/bit) is **0.37**. A 0.9 gate would never have fired on the very report it was written for. Measured across mismatches: **0.34-0.50 for a wrong rate against ~0.004 for a healthy capture**, so a quarter separates them with room on both sides.

force_conflict defaults to adopt

'adopt' takes the detected rate, rounded to a credible boundary, locks it and says so on the note row. 'ask' refuses the decode, names the rate the wire carries, and leaves the choice with the operator: `sdec.rate_accept()` takes the detected rate, `sdec.rate_decline()` keeps theirs.

Adopt is the default because the refusal could not be answered from the panel. Measured on the instrument: BAUD read '16099?' locked, the note read '18700 baud fits nothing; this capture reads 16099', no bytes, and no control offered the detected rate -- `rate_accept()` has no button, and 'Lock Rate' hides itself while a rate is locked, which it was. The only route out was Options and the number typed by hand, which is also how the wrong rate got there.

The case against adopting was that it puts bytes on the panel at a rate nobody chose. What answers it is the note row plus the gates: the rate is adopted only when it has been decoded at successfully, the operator's own rate failed, and the wire's measured bit time disagreed with it. A rate that explains the capture is not 'a rate nobody chose'.

Nothing blocks on a human in either mode. There is no modal wait -- the offer is published as state and the two answers are ordinary functions -- so a headless caller drives either branch and an unattended lap that never answers stays refused.

The relock retry has to earn it

The gate that keeps a correct hand-typed rate safe. `rate_note` is not proof the rate is wrong: its integer-multiple branch fires on healthy captures too, `sig_bittime` fitting the greatest common period. So a right rate with a wrong forced format -- errors everywhere, plus that artefact -- would otherwise have its rate thrown away and a detected one substituted. Raised by review.

So: unlock, re-decode, and keep the result only if it beats the threshold that triggered the retry. Otherwise put the operator's rate back and decode at it again -- three decodes in the failing case, which is a frame capture's cost and nothing on the streaming path. There is no restore path on the unlock itself: the lock explained nothing, so being left unlocked is the useful outcome even if detection does no better.

Adopting a rate from the measurement when the snap was not firm

`found` is `sdec.baud`, which `sig_snap` has already relabelled, so adopting it relocks the overstated standard rate and makes a one-capture mislabel permanent. Cluster C reached the instrument through exactly this shape. `autolock_try` uses `baud_raw` for the same reason.

The adopted value is rounded so that what gets locked reads like a setting: 16099 locks as 16100. Not rounded when the value is a standard rate, and not rounded when the shift would exceed 1 % -- `baud_round` decides both and hands back `credible` for the second case. Locking it rather than leaving the app unlocked for autolock to pick up: autolock is skipped for one capture after any request for auto-detect and has four more gates of its own, and leaving it unlocked is what made the panel read 'unlocked' while the recording modes stayed unreachable for want of a locked rate.

Why the hex row is two groups of eight

FONT_SMALL is not a fixed-pitch face: hex digits measure **~7.47 px against ~8 for a digit column**, so a 16-byte row's characters drift relative to any grid as the row is read, and by the far end a byte is most of a character-width away from where the eye expects it. The firmware offers no monospace font, but the drift is cumulative, so two eight-byte groups halve the worst case and the gap gives the eye a hard restart point instead of one 47-character run to track across. Eight is also the natural group for the job: a byte offset within a row becomes 'left half or right half, then count at most eight', which is how anyone reads a hex dump. Both formatters group identically, since a log file whose rows do not match the panel's rows is a file you cannot check the panel against.

On screen the two groups are separate display objects at fixed x. One string with the groups spaced apart inside it does not work on this panel: where the second group starts depends on the rendered width of the first eight bytes, so '00 00 ...' and 'FF FF ...' are different widths and the second column wandered row by row. The ASCII field is not split to match -- its job is to be read as text, and a gap in the middle of a string breaks words for a column correspondence the proportional font cannot deliver.

Cells are two characters joined by a separator, not '%02X ' with a trailing space. The trailing version is one character wider than the field it lives in -- **48 rather than 47** -- and since the vertical rule's x is computed from `ui_hxn * ui_ch_w`, that one space closed the gap to exactly 0 px and the last byte rendered on top of the rule. Visible in the mockup as '...33 35' touching the divider, and easy to mistake for a font-metric problem rather than an off-by-one in the string.

The file form keeps its '|ascii|' delimiters, a log line having to be readable and parseable on its own, where the pipes are the only thing marking the ASCII field. The screen form drops them because vertical rule objects do that job at a fixed x, which a '|' inside the string cannot -- it sits wherever the preceding hex happened to render. The hex block also loses the file form's leading space: there it separates the offset from the data inside one string, here the offset is a different object and the space would push the block off its own column by one character.

ua_hexrow is total for any vals content

Load-bearing rather than defensive: `ck_sink_file`'s `emit()` formats a row and then shifts its carry, so a raise out of the formatter leaves cn and cbase describing a row the file never got, while `flog_bytes` has already counted those bytes on acceptance. Making it total is what lets both call sites run it bare; the alternative was a pcall around our own formatter at each of them.

A type test alone is not enough, and the gap is interpreter-dependent. '%02X' on a number with no integer representation **raises under the host's 5.1+ Lua** (1.5, nan, inf, 1e300) and merely **casts under the 5.0.2 that ships**, so a type test passes every offline suite while the totality claim is false on the interpreter the tests run on. The range test is what makes it true on both, and it also keeps the row fixed width: 256 or 65536 formats without raising and widens the row, which no test would have caught either.

Measured on the host: a string, a boolean and a table each raise -- the first two words through `string.format('%02X')`, a numeric string through `ua_char`'s `v >= 32` -- and so does any number without an integer representation. `v ~= v` is the nan test; floor(inf) == inf, so inf is caught by the bound rather than by it. A nil already rendered as ' --', so every rejected value joins a branch that already existed.

tmin0 makes ua_run resumable

`chunk_decode.tsp` passes the position (in this array's coordinates) at which the previous frame ended, and the anchor search is skipped: the first frame in the window is then found the way every subsequent one is, as the first candidate edge not inside its predecessor. Frames before `tmin0` are not emitted at all, which is simultaneously the continuation rule and the de-duplication rule for an overlapping window.

It has to be a parameter rather than a second copy of the loop. The sequential walk is the subtlest code in the app, and a chunked variant of it would drift out of step with this one at the first bug fix.

Resuming also beats re-deriving the phase. On a gapless stream the 1.5-bit anchor search falls through to the first candidate edge, which inside a byte is a data transition, and every frame after it decodes misaligned.

The polarity contest needs a margin

A raw score comparison lets the frame-count bias in by the back door: the losing polarity can win its own search with a shorter frame that finds more frames. On repeated 0x55 **the inverted 6N1 scores 11 against the correct 8N1's 10**, flipping polarity and width at once. So the prior polarity goes first and keeps the decision unless the other clears the frame-shape margin, and a forced polarity ends the contest rather than entering it -- otherwise a forced setting gets overturned by a score. How hard the prior is to overturn depends on where it came from: see 'polmargin = 4, and the inverted-ASCII trap'.

math.mod, not math.fmod

5.0.2 has no modulo operator and no `math.fmod`. The offline suites shim `math.mod` to `math.fmod` (`tools/gen_serial.lua:22`), so a call written `math.fmod` passes every offline test and dies on the instrument: measured, that build refused every capture with **"attempt to call field `fmod' (a nil value)"**. No lint catches it, because the shim makes the wrong spelling valid everywhere the tests run.

An inadmissible fit on an undersampled line

`sc1` is nil exactly when the width fit failed one of its own two admission tests -- `ua_plausible` (samples per bit inside the range this instrument can carry) or the relative q gate. That is what rejects an undersampled line: at **921600 baud on a 1 MS/s capture the fit is 1.09 samples per bit**, and four plausible-looking bytes once came back labelled 230400 baud.

Pulse-width evidence is real evidence, so when no rescaling frames anything the fit is kept -- unless it is not even admissible, which is the one case where returning no bit time at all is the honest answer.

Acquisition

The sample-rate ladder divides 66 MHz

Measured over 26 rates: the sample clock is 66 MHz with an integer divider, rounded up, so `fs_actual = 66e6 / ceil(66e6 / fs_requested)`. The model predicts all 25 accepted rates to within **4.3e-7**, so a rate is exact if and only if 66e6 is divisible by it. 66e6 = 2^7 * 3 * 5^6 * 11, so 120000 is exact and 160000 =

2^8*5^4 is not -- it wants one more factor of two than the clock has.

1000 is the floor; 900 is rejected outright with error 1130.

Three inexact rates are listed anyway -- 160000, 320000 and 640000, off by -0.121 %, -0.362 % and -0.841 %. They are what 19200, 38400 and 76800 need: without them those three, the commonest rates on a bit-banged micro, snap up a whole step and get 192, 192 and 153 bytes per capture instead of 240 each. Safe because the shortfall is downward and tiny -- 8.32, 8.30 and 8.26 samples/bit, all above the 8 the selector asked for -- and every bit time is scaled by `acq_fs` rather than the requested rate. 350000 is omitted only because no standard baud rate wants it.

Why 153600, 172800 and 192000 are in stdbaud

The ladder jumped 128000 -> 230400, so a 153.6 kBd line (8 x 19200, the next doubling anyone sweeping upward reaches) could not snap: the nearest listed rate is 20 % away against a 2 % `snaptol`. Nothing decoded wrongly, but `autolock_try`'s first gate is `snapped`, so the rate was never pinned, the recording modes stayed unarmed, and the only way in was typing the raw figure into Options. Pinned by hand, an 8 kB recording at **153 588 baud captured and decoded correctly**.

172800 (9x) and 192000 (10x) were found the same way -- a 192 kBd line reported a raw 191 993 with a question mark. Spacing against the 2 % snaptol: 153600 -> 172800 is 12.5 %, 172800 -> 192000 is 10 %, 192000 -> 230400 is 17 %. No new ambiguity.

307200 is not listed, `maxbaud` being 250000: adding a rate the detector refuses would be listing a refusal.

Which of these can record follows from `fs_for_burst`: the burst path loses `acq_overhead` per sample, so the crossover is 1/(4*(1/1e6 + 5.1e-7)) = **165 563 Bd**. 153600 is the fastest standard rate that can record, at 4.31 samples/bit delivered; 172800 and above are frame-only. Frame decodes to the full 250000.

Why the levels are the two densest amplitudes

Working inward from the extremes is what a percentile trim does, and it fails because the trim fraction must be guessed against an unknown outlier count. Measured: eight 2-sample 25 V spikes on a 0/3.3 V line are **16 samples in 1400, 1.1 %, over a 0.5 % budget** -- so the trimmed upper bound landed inside the spikes, the high-group mean was dragged to ~12 V, the threshold went with it, and the edge list collapsed **from 80 edges to 12**. The decoder then fitted the 2-sample spike pairs and reported **47 427 baud with six confident-looking bytes**. Perversely, forty spikes worked, because past the trim budget the bound landed outside them again.

Density has no such budget: a logic level holds hundreds of samples in one or two bins, while spikes of differing amplitude scatter one or two samples per bin, and even a wide spike train loses to the minority logic level on count.

binocc = 16

A fixed 512 bins over a 2000-sample probe averages four samples a bin, and at that occupancy Poisson scatter alone drops the occasional bin below any sensible threshold. One such dip inside the idle hump ended the width walk early, left the rest of the same hump looking like a separate level, and put **both levels within 0.35 V of each other on a 3.3 V line**. Sixteen samples a bin keeps the scatter far below the hump threshold.

The flow-control pulse is ~10 us, not the 100 us requested

`trigger.extout.pulsewidth` is nil on 1.7.17a, so `fc_config` skips the assignment and the width is the firmware default. Measured on an SDS1204X-E at 1 GSa/s, armed SINGLE on the rising edge: **9.41 us and 9.82 us** on two trials, 4.92 V peak, -0.96 V baseline.

So a device waiting for a credit needs an edge-triggered input, an interrupt or a latch: 10 us is 2.5 bit times at 250 kBd, and a firmware polling a GPIO in a loop can miss it entirely. The 100 us request is kept because it is the right request -- 25 bit times at 250 kBd, comfortable even on a 1 MHz part -- and a firmware that does expose `pulsewidth` will get it.

trigext_only

`trigext` ORs the rear input with the analog start-bit trigger, which is the right default. But on a line that is always transmitting the start bit always wins, so an external marker can never decide where a capture begins. Measured: with the marker off and the line quiet the capture **refuses in 6 s**, and with the line live it completes immediately on the start bit whether the marker is there or not.

armlevel 1.0, floor 0.33, ceiling 6.0

1.0 clears ADC noise and the slow edge of a lightly-driven line and still sits inside the swing of every family decoded -- a 3.3 V part crosses it at under a third of its swing from either direction. Below 0.33 V the comparator starts firing on noise on a floating input, arming the capture on nothing. The 6 V ceiling is because a level must sit inside a logic swing to be crossed at all, and a 6 V arm already needs a 12 V part.

On a line idling away from ground the level is clamped at the midpoint: the active level is unknown by construction, so half the idle level is the deepest arm certainly still inside the swing.

This equals `flatfloor` by coincidence, not construction. `flatfloor` decides whether an idle level counts as sitting at ground; this decides how far above such a line to arm. Unifying "the two 1.0s" would couple the operator's trigger level to the ground test.

armwait 10, floor 2, ceiling 120

10 rather than the 3 an 'edge' capture waits, because the wait has to outlast an operator reaching for a power switch. **The panel is frozen for the whole wait**: the loop runs inside a touch handler and nothing can interrupt it -- the front TRIGGER key's event never reaches the cancel latch while a panel-initiated run executes, and a touch press only queues. At the 120 s ceiling that is two minutes of dead panel, which is why `armnote` exists. Raise the ceiling no further without making the wait yield.

The 2 s floor is why nothing pre-existing changes: the caller takes this value only when it exceeds the mode's own wait, and 2 against an 'edge' default of 3 means a form at its minimum leaves every pre-existing timing as it was.

Acquisition configuration

The 10 V range has the widest digitize bandwidth

Fixed at 10 V and not offered as an option. Digitize bandwidth by range: **440 kHz at 10 V**, 210 kHz at 1 V and 100 mV, and only **17 kHz at 100 V and 1000 V**. Nothing worth decoding swings under 1 V, and anything swinging over 10 V wants a divider (`docs/lin-divider-panel.png`), so the widest range is also the simplest choice.

Sample rate follows a forced baud rate

`sdec.n` is 20 000 samples, so at a fixed 1 MSa/s a capture is 20 ms of signal whatever the line speed -- which makes a slow line yield **less** text. At 9600 8N1 that is 19 bytes. A forced baud rate is known before the capture, so the rate can be chosen deterministically:

forced baud	fs	samples/bit	bytes
1200	10 k	8.33	240
2400	20 k	8.33	240
4800	40 k	8.33	240
9600	80 k	8.33	240
19200	160 k	8.33	240
38400	320 k	8.33	240
57600	500 k	8.68	230
115200	1 M	8.68	230 (at the ceiling)
250000	1 M	4.00	500

The rate must be snapped to `sdec.rates`: only decimal rates are exact, and asking for 102400 gives 102 325.5 Hz. 8 samples/bit is the target because 8.68 is already a proven case in the suite.

The blind arm commits to 38400, and the choice is a window length

An arm from silence cannot measure the bit time first -- nothing is on the line -- and it gets one attempt, because the window holding a device's opening bytes is the only window those bytes are ever in. The arm fires on the first edge whatever the rate, so every rate below catches the start; what changes is how much of the opening message comes with it (8N1, at `sdec.n`):

fs	window	9600	19200	38400	57600
160 kS/s	125 ms	16.7 / 120	8.3 / 240	4.2 / 480	2.8 -- refused
200 kS/s	100 ms	20.8 / 96	10.4 / 192	5.2 / 384	3.5 -- refused
320 kS/s	63 ms	33.3 / 60	16.7 / 120	8.3 / 240	5.6 / 360
500 kS/s	40 ms	52.1 / 38	26.0 / 77	13.0 / 154	8.7 / 230

samples per bit / bytes of window. 38400 at one sample over the floor is 200 kS/s; everything else is spent on window, because the rates a bench meets are overwhelmingly 9600 and 19200 8N1 -- a prior worth about 90 %, and nothing below 9600 at all. A hundred milliseconds holds 96 bytes of a 9600 Bd power-on banner against 38 at 500 kS/s.

A faster line loses only its opening bytes: `capture`'s back-off re-runs `autoset`, and by then the device is transmitting, so the ordinary probe measures it properly. `sdec.arm_try` fits the standard rates to the samples already in hand, which costs nothing and loses nothing.

Rates and the burst path

acq_overhead = 0.510 us

A SimpleLoop trigger model adds a **constant 0.510 us to every sample interval** -- dead constant from 1250 S/s to 640 kS/s, and independent of aperture (AUTO and 1 us are identical). `dmm.digitize.read()` does not pay it and hits requested rates exactly, but it blocks Lua for the whole acquisition, so the idle watchdog cannot poll `buf.n`; chunking it to regain control costs **9139 us per seam**, 8.8 bytes at 9600 baud. The streaming modes keep SimpleLoop: an offset rate is measured and compensated by `acq_measure_fs`, whereas a seam is data that is gone.

This is also why an exact-rate table measured with `digitize.read()` does not describe this app.

fs_for_burst delivers a floor rather than requesting one

Solving `fs_actual = 1/(1/fs + 0V) >= baud * want` for the requested fs gives `fs >= 1 / (1/(baud*want) - 0V)`. Without it the burst path asked for exactly `baud*minsabit` and delivered less: **115200 asked 500 kS/s for 4.34 samples/bit and got 398 kS/s and 3.46** -- under the declared floor, while the app believed it had cleared it. It appears only on hardware; offline, ladder snapping made `minsabit` look safe.

`fs_for_baud` maximises oversampling, which is right for frame mode where the format is unknown. In a streaming mode baud, width, parity, stop bits and polarity are all forced, so the margin buys nothing and the samples cost everything. At the predicted 8 us read:

baud	fs_for_baud	fs_for_burst	delivered	gain
115200	1 M, 8.68 sa/bit	640 k	4.19 sa/bit	1440 -> 2880 B/s, 32 kB -> 64 kB/buffer
38400	320 k, 8.33	200 k	4.73	1229 -> 2400 B/s
9600	80 k, 8.33	40 k	4.08	1501 -> 2400 B/s

115200 was the worst baud in the table -- slower to decode than 9600 -- purely because the ladder handed it 8.68 samples/bit. Both wins reduce lost data: fewer samples per byte means more bytes fit one buffer and a shorter decode between windows.

Asking for 5 instead of `minsabit` gains nothing, the ladder being coarse: 115200 x 5 = 576000 snaps back to 1 MS/s, the rate this is trying to avoid.

The no-loss sample rate is 120 kS/s, and baud is not in it

Decode cost is per sample, so the ceiling is 1/(framebits x T x tread) bytes a second while the wire produces baud/framebits. Setting ceiling >= wire cancels framebits **and** baud and leaves `delivered sample rate <= 1 / tread`, about **120 kS/s at the measured 8.3 us a sample**. That is the fastest a recording can sample at any baud and still have the decoder keep up, and it is where both `minsabit_tier` entries come from:

baud	floor	fs	delivered	decode ceiling	wire	tolerates
9600	4	40 k	4.08	3061 B/s	960 B/s	no loss
9600	8	80 k	8.01	1561 B/s	960 B/s	no loss -- free, and the three-tap vote switches on

baud	floor	fs	delivered	decode ceiling	wire	tolerates
19200	4	80 k	4.00	3122 B/s	1920 B/s	no loss, zero margin
19200	5	120 k	5.89	--	1920 B/s	the most 19200 can have for nothing
19200	6	160 k	7.70	1623 B/s	1920 B/s	85 % duty
19200	8	200 k	9.45	1322 B/s	1920 B/s	69 % duty

At 9600 and below the decoder outruns the line with 8 samples a bit, so the oversampling is free. That matters because 4 samples a bit has no room for a rate error: the bit centre drifts by err x bit_index, so an eighth of a bit is spent after 1/(8*err) bits -- one byte at a 1 % error.

19200 takes the margin and not the vote, deliberately. Asking 6 lands on 160 kS/s and 7.70, which crosses `ua_bit`'s T >= 6 gate but delivers 147 929 S/s -- past the no-loss rate, so a line transmitting more than ~85 % of the time starts losing data, and a boot-log dump is exactly that. 5.89 is 47 % more samples a bit than the bare floor at no cost. The taps at 5.89 land +/-1 sample out, 17 % of the cell, where 7.70 rounds to +/-2 and lands at 26 % -- so the gate excludes a placement better than the first one it admits, and relaxing it to 5.5 would give 19200 the vote for nothing. That is a decoder-wide change and wants hardware behind it.

Nothing above 19200 has a tier: 38400 would have to sample at 200 kS/s to hold 4 samples a bit, which is already past the no-loss rate. The loss there is unavoidable and `minsabit` is what bounds it.

The reading-backlog dialog above ~100 kS/s

Above about 100 kS/s the instrument puts its own dialog over the capture: "Processing reading backlog -- the digitizer is producing readings faster than the meter is moving them out of the buffer". Operator-reported, it clears by itself and the recording carries on normally, so the `fs_cap_tier` cap buys an uninterrupted display and nothing else.

It is a rate condition, so every rate above ~100 kS/s crosses it and each has to justify itself:

- **At and below 19200, avoided.** 100 kS/s delivers 9.91 samples a bit at 9600 and 4.96 at 19200, and nothing faster buys anything the decoder reads. 120 kS/s for 5.89 at 19200 would cross the threshold to gain 0.93 samples a bit and still not reach the vote -- a dialog for nothing.
- **38400 and above, unavoided, and it earns it.** 200 kS/s to 1 MS/s is what holds four samples a bit at all, and capping it would make every line above ~23 786 Bd unrecordable.

The cap is a hard maximum and may override the floor, which is the one place a floor gives way: below the cap at 19200 only 4.96 samples a bit is available, under the tier's 5, and taking it is still 24 % more than the bare floor. It never gives way past `minsabit`.

What would refine this is one observation rather than a redesign. A backlog is excess rate times duration, so a short capture can outrun the threshold without reaching it: the 8 kB mode at 19200 runs 8192 x 10 / 19200 = 4.3 s, which at 120 kS/s accumulates only ~55 000 readings of excess against the 32 kB mode's 17 s and four times that. If the notice does not appear for the 8 kB mode at 120 kS/s the cap should be duration-aware and that mode can have 5.89 samples a bit for nothing. Where the threshold actually sits is unmeasured.

sabit_max = 12

`ua_bit` reads three samples a bit whatever is available -- the centre and +/-0.2 T -- so past the vote threshold extra samples are never consulted and buy only bit-centre placement, +/-0.5/T of a bit, while costing readings and a decode priced per sample. At 8 samples/bit the placement error is **6 % of a bit and at 12 it is 4 %**, against a frame that tolerates far more.

What the ck_bufmax bound on oversampling is worth

A byte-capped mode holds cap x framebits x samples_per_bit readings, so oversampling past what `ck_bufmax` allows does not fail -- `stream_begin` pins the capacity and shortens the recording instead, **measured at 26.9 kB of a nominal 32 kB** where the ceiling bound the capacity. So the ceiling is per mode, not per baud.

Enumerating every combination that returns a rate -- 18 standard bauds x {8 kB, 32 kB} x {edge, free, front} x forced width, parity and stop bits, 8208 of them -- and counting those where asking with the mode differs from asking without it:

ceiling	combinations bound
4 076 503 (shipped, defbuffers at their defaults)	306 of 8208, every one the 32 kB mode with a frame forced wider than ten bits
3 700 000	562, and at 110 Bd it binds at 8N1 too
930 255 (a 6 000 000 `defbuffer1` left in place)	1858, including the 8 kB mode

So at the shipped ceiling the bound reaches only forced-wide frames, and it is not therefore dead weight: the figure `start()` derives falls as the operator's own buffers grow, and the lower it falls the more ordinary the combinations it holds down. It is insurance against the ceiling the instrument actually leaves, not the one shipped here.

`fs_for_burst`'s bound is `stream_begin`'s own clamp inverted, not a second approximation of it. `stream_begin` sizes the capacity as ceil(n x 100 / (100 - pretrig)) + 100, so the largest count that fits is (ck_bufmax - 100) x (100 - pretrig) / 100. Dividing by (100 + pretrig) instead is close but larger -- 0.95238 of the ceiling against 0.95 -- so the bound admitted counts the clamp then pinned, and for any ceiling in a ~9000-reading window the two disagreed about whether a rate fitted at all.

Capture geometry

window_bytes: one division, not two

The obvious form, `floor(n / (fs/baud * framebits))`, rounds twice: 10000/1200 is 8.3333333333333339, ten of those is 83.333333333333343, and 20000 divided by that is **239.99999999999997** -- so it floored to 239 where the exact answer is 240. `n * baud / (fs * framebits)` is algebraically identical with a single rounding step, and exact whenever fs is an exact integer rate, which is the entire reason `sdec.rates` exists. Both products stay far inside double precision: the largest case is a 4.1 M-sample buffer at 250 kBd, 1e12, against 2^53.

Worth fixing rather than shrugging at, because the whole point of the number is "was this message seen whole?", and every case where the division comes out exact is a case where the operator was told the window was one byte smaller than it is.

The pre-trigger reserve comes out of capacity, not out of count

`LoopUntilEvent` programs (100 - position)% of the **buffer's capacity** as its post-trigger count, and `dmm.digitize.count` has no part in it: measured over 15 combinations of capacity and position, each matching that rule exactly, with `digitize.count` set to 77 against 1000 changing nothing. The shorter rule the vendor manual implies -- n minus n x position/100 -- under-reports by 5 % of the window on every armed capture.

Two measurements at two capacities both fit, which makes it a rule rather than a fit to one point:

capacity	post-trigger count	delivered
20 000 (a buffer sized exactly n)	19 000	19 011; a dump held 227 bytes under a panel claiming 240
21 100 (the reserve asked for on top)	20 045	20 066

So `acq_cap` asks for the reserve on top of the depth and an armed capture delivers the full depth -- measured 20 050 to 20 350 readings. `n_deliv` is a **floor, never a ceiling**: the pre-trigger samples accumulated before the trigger arrive on top of it and are unknowable in advance, a handful on a busy line and up to the full reserve on one that was silent. It comes out above `sdec.n` because 95 % of 21 100 is more than 20 000; the 100 readings of headroom in that capacity are the only part not reported, since they exist to absorb the model's overshoot.

The headroom is unconditional, including at pretrig = 0: the model can make a reading or two past `count` before `abort()` takes effect, and with a buffer sized exactly n that overshoot is a discard. Measured with pretrig = 0 and no headroom, **20-30 events per capture**.

A reserve in a FILL_ONCE buffer posts 4915 per discard

`buffer.make()` gives a FILL_ONCE buffer, where a reading past the end is discarded and every discard posts **4915 "attempting to store past the capacity of reading buffer"** at error severity.
`LoopUntilEvent`'s pre-trigger reserve is inherently circular -- it keeps the last readings before the trigger, so it must overwrite its oldest while it waits -- and on a FILL_ONCE buffer that is a discard. It fires whenever the first edge is further off than the reserve holds in time:

fs	reserve	got	4915
1 MS/s	1.05 ms	21 100 (the whole buffer)	x10
10 kS/s	105 ms	20 085 (the depth asked)	x0

Measured at 1200 baud, one armed capture each; ten per capture at 300-1200 baud and none at 9600 and up. 12.7 ms at 9600 baud against 0.42 s at 300.

Not fixed by raising `position`: a reserve big enough in time at 1 MS/s is 66 %, which would cost two thirds of the depth. Nor by a bigger buffer -- 5 % of enough readings to cover 40 ms at 1 MS/s is an 800 000-reading buffer per capture. The fill mode is what stops it. `buffer.FILL_CONTINUOUS` the **constant** does not exist on 1.7.17a and assigning it raises, but the **attribute** takes the numeric value: `fillmode = 1` reads back as 1, `defbuffer1` ships in that mode, and an armed capture in it posts no 4915 at all -- measured at 1 MS/s on an idle line, two events for the whole capture and neither of them 4915. The data is unaffected either way, verified byte-exact.

The mute does not cover it. With `localnode.showevents = 0` in force, an armed FILL_ONCE capture still puts a modal Error 4915 box over whatever is displayed. The same mute does suppress 1130 and 2205, so this is a subsystem difference and not a severity rule. Measured and photographed: `docs/vendor/02-event-4915-unmutable.md`.

A record can come back rotated

The capture buffer is FILL_CONTINUOUS and `LoopUntilEvent`'s pre-trigger block writes into it for the whole wait, so an arm that waits seconds laps the buffer: 2.6 s at 200 kS/s is 24 laps of 21 100 readings. `readings[1]` is then the ring's physical slot 1 rather than its logical oldest.

Measured on 1.7.17a, an armed capture of a line that started talking 3.1 s after Capture: **`buf.startindex` 13 288, `buf.endindex` 13 287**, and the 1055-reading pre-trigger silence sitting at 13 288 instead of at 1. The dump opened on 'World!Hello, World!...' -- mid-message, with one framing error where the ring seam fell. Every byte was present and the order was wrong.

`bufferVar.startindex` answers it in one proxy read. A timestamp binary search would also find the seam, the timestamps being monotonic in logical order, but that costs fifteen reads for a question the instrument answers in one. Not rotated is the normal case and costs the same one read: a capture that never filled the buffer, a free-run read, and an armed capture of a busy line all come back with startindex 1, the trigger firing within about fifteen samples of `initiate()` on a line already transmitting.

`buf.n` is no substitute as a progress counter: it stops at capacity, so on a FILL_CONTINUOUS buffer it says "full" and then nothing for the rest of the acquisition -- and an armed recording whose pre-trigger phase waited longer than the buffer holds is in that state before the device has sent a byte, 0.75 s of wait at 1 MS/s filling an 8 kB recording's buffer. `endindex` keeps moving.

buffer.STYLE_COMPACT is nil on this firmware

STANDARD, FULL, WRITABLE and WRITABLE_FULL all resolve; COMPACT does not. The style constants are opaque userdata rather than values, so it cannot be passed by value either -- `buffer.make` refuses the string form even for a style that does exist, which is the control proving the route closed rather than compact rejected. It would buy capacity and measurably no decode speed, the read cost being payload-independent. The capacity it would buy is **20 000 000 readings against the 7 500 000 standard ceiling on a DMM7510** -- both from that instrument's reference manual at 10702, neither measured here, and the compact figure unreachable on this firmware in any case. See 'The reading pool is per model' for the attributed three-model table.

acq_fit takes a percentage of the buffer, with no fudge term

The clamp takes the reserve as a percentage of **this** buffer, which is what the model takes, rather than of the capture intended -- the two differ exactly when the buffer came out smaller than asked for, which is the case the clamp exists for. `acq_cap` sizes the buffer as n + reserve, so room comes out at n and a capture asking for its own depth is never clamped. An extra -1 cost a sample on every armed capture and truncated the last frame.

Signal conditioning

Hump membership is the bin index

Membership is recomputed with the identical expression the histogram pass used, not by re-deriving each hump's voltage band as `v0 + (a−1)/scale` and comparing volts. That round trip lost in production: a clean 0 / 3.300 V capture, 2272 samples, so nb = 142 and scale = 141/3.3. The tallest hump is the single bin 142 holding vmax itself, and its re-derived lower edge came out **4.44e-16 V above vmax -- 1.9e-14 of a bin**. Not one sample fell in the band, na was 0, and `sig_pair` returned "line is idle (no transitions)" on a textbook waveform.

It is **length-dependent**, which is what makes it nasty rather than merely wrong: the same waveform at 13, 14, 16 and 17 bytes round-trips exactly and only 15 bytes fails, so it presents as one capture in five inexplicably seeing a dead line -- indistinguishable at the bench from a probe on the wrong pin. Comparing bins cannot drift, the humps having been chosen by bin index. It also fixes a quieter inconsistency: the voltage test was inclusive at both ends while the histogram assigns by `floor()`, i.e. half-open, so a sample sitting exactly on the boundary between the humps could be averaged into the wrong one.

Carrying each bin's **sum** alongside its count makes the level means a walk over bins rather than a second walk over the capture -- 512 iterations against 32 768 at the largest capture size. Three ops per sample buy a whole pass. The two humps are disjoint by construction, m2 being chosen outside [a1, b1] with each expansion stopping before re-entering it, so the ranges are summed independently rather than through the first-hump-wins test a per-sample pass needs.

Summation order changes, and that is the one thing here not bit-identical: a per-bin subtotal added into a running total rounds differently from 32 768 samples added in capture order. It is the more accurate of the two -- 512 partial sums is a flatter reduction tree than one long chain -- but the levels can move in their last bits and `sdec.thr` with them.

minhump = 2

The minimum that kills a one-sample hump, and deliberately not the 16 this first shipped as. `binocc` is a per-bin occupancy target, the count at which Poisson scatter inside one bin stops mattering; `nbb` is a whole hump's total, so using one for the other was a category error.

Sweeping the gate from 2 to 16 changes nothing about noise rejection: **31 of 40 uniform-noise seeds are refused at every value in that range, and gaussian-shaped noise is refused at none of them**. So this gate is not what refuses noise; the hump-swallowing test is, where it applies, and everything else is caught by `modal < minmodal` or by `sig_bittime` finding no usable pulse widths. A large gate buys no noise rejection and costs real signals: at 16 it refused a sparse burst -- one short message in a long recording, the traffic the streaming modes exist for -- because a decimated view of it holds only a few samples at the space level.

Noise is excluded by position, not by amplitude

A sample partway up an edge is indistinguishable from a noisy one by amplitude, so an amplitude window cannot separate them. Measured on a noiseless capture: a thr+-hyst window reports **24.8 dB** and a within-15 %-of-level window **38.4 dB**, both of them pure slew, and neither moves when real noise is injected 40 dB below that. Requiring both neighbours on the same side drops everything within one sample of a transition and tracks injected noise to **0.3 dB from -24.8 dB down to -84.8**. It costs 23 % of the samples.

The sampling is strided rather than the first N: a thousand contiguous samples at 1 MS/s span 1 ms, a twentieth of a 50 Hz cycle, so mains pickup inside that window is a slow tilt that reads as signal -- and at the head of a capture they can all land in the opening idle, where only one level exists. Striding the whole capture costs the same and sees both. Samples in transit are excluded or the answer is dominated by them, there being about two per bit sitting nowhere near either level. Residuals are taken about each group's own mean and pooled; measuring about `thr` would report half the swing as noise.

idlemult = 10.5

How long a run must be before `sig_idle` believes it is idle, in estimated bit times. At 8N1 the longest run data can make is nine bit times -- a start bit plus eight zero data bits, or eight ones plus the stop bit -- so ten is the smallest whole number that must be idle. Ten is still too tight, because both terms carry error: the run is measured on a jittered edge and one bit time is a low percentile of the pulse widths.

Measured on 1024 random bytes with bit 7 cleared at 10 % jitter and 2 % noise: **the longest run is 90.88 samples against a threshold of 90.7**, so the guard misses by 0.2 % and `sig_idle` takes its run-length branch on a window holding no idle at all. There the longest run is a low data run, so idle comes back 0, `ua_autoformat` takes the inverted reading as its incumbent, and the capture decodes self-consistently with every byte wrong. 10.5 restores the margin: the sweep in `tools/sweep_all.py` goes from **7 silent wrong answers to 0** and from 2307 exact decodes to 2314, at a cost of one more capture landing in the 7E1/8N1 ambiguity it counts separately. Reverting to 10 puts all seven back, measured both ways.

A cleared top bit is what makes this reachable, and it is why the paired vectors exist: v93 and v97 are the same 1024 bytes and only v97 has bit 7 masked off. Clearing it removes the long high runs, so the longest run in a window becomes a low one. **v93 inverts in 0 of 24 captures, v97 in 8 of 24.**

This is not what inverts captures on the bench, and the distinction matters because the two look identical in a record. This constant guards the run-length branch. Every inverted cell in a 258-cell hardware diagnostic took the **levels** branch, at longest/onebit ~ 6 against a guard at 10.5, so no value of this constant could have reached them -- their cause is a ground-straddling wire, which the levels branch reads as RS-232 and is right to. Two mechanisms, one fix each.

sig_idle's run branch, and when the levels decide instead

A run is only evidence of idle if data could not have produced it. Measured at the bench on real 9600-baud ASCII into the front terminals: **run0 = 63 samples, run1 = 42**, so longest-run said idle = 0 on a line that idles high. Both runs are data -- 63 samples at 10.42 samples/bit is six bits, a space (0x20) whose start bit plus five zero data bits run together, and 42 is the four 1-bits of a lowercase letter. The window held no idle at all.

That is worse than a coin toss because `ua_autoformat` makes the prior its incumbent and requires a rival polarity to beat it by 60 %, and the inverted reading of ASCII frames just as cleanly as the correct one: ASCII's top data bit is always 0, which puts a rising edge exactly nine bit times after every start bit -- a perfectly periodic fake start edge. At 57600 baud the inverted reading scored **218 good frames and 0 bad**.

On an armed capture the leading run is never evidence, because it is not the line: it is the pre-trigger reserve, the wire before the device started. On a single-supply line that is ground, and ground is the space level, so it arrives as a run of lows no data could produce. Measured, v41 (a 0.00..3.30 V line) armed on silence with the generator switched on at +3 s, six captures a condition:

condition	run0	run1	idle	result
busy line, no arm	50	344	1	162 bytes, 0 bad

condition	run0	run1	idle	result
armed, fs 200 kS/s	1052	1210	1	62 bytes, 0 bad
armed, fs 80 kS/s	1053	500	0	158 bytes, 62 bad

The third row is the defect and the second shows why it is not always reached: the reserve is a fixed **sample** count while the line's real idle gap is a fixed **time**, so the gap only out-runs the reserve above about 171 kS/s on this vector. Below that the reserve is the longest run in the window, the prior becomes 'idles low', and `ua_autoformat` searches inverted first. The length is also what disarms the protection built for this: a run that long clears idlemult x onebit easily, so the levels branch never runs and `idle_weak` stays false -- which is the flag `polmargin` keys on. The capture that most needs the 4x margin is the one certain not to get it.

Dropping the leading run is not enough on its own, because what is left may be no better. Idle recurs and an artefact does not: a line's idle level appears in every gap between messages, so a window holding real idle holds several interior runs past the idle threshold at one level. A break appears once, and by length alone a break cannot be told from idle, both being longer than any run data can make. Measured, an RS-232 session of gapless frames ending in a break: **lead 20 830 samples at mark against a 2083-sample break at space**, and with the lead dropped the break became the longest run and flipped the prior. The break is interior, so excluding the tail instead would not have helped. Two interior runs are the bar; one is an excursion.

Without recurring interior idle, decide from the levels. That is a fact about signalling rather than a guess about the window, and it is right in both cases the runs cannot settle: gapless frames on a single-supply line mark high, and the gapless-plus-break RS-232 session straddles ground and marks at its negative level. It is wrong for an inverted TTL line with no idle in the window -- MIDI's current-loop receiver side, or a deliberately inverted debug pin -- and that case keeps working whenever the window does contain idle, with Options -> Invert forcing it otherwise.

The run branch still decides wherever it can. A window with recurring interior idle takes it, so an inverted single-supply line -- idle on the bottom rail, which the levels would get wrong -- keeps decoding as long as its gaps are in the window. Measured on such a line, armed, at three bands and two lock states: **36 of 36 captures correct**, idle 0, invert true.

Bit timing

math.abs was 14.2 % of every C call the app makes

Measured over 300 bench cells, in `sig_fit`'s fit loop -- the hottest counted loop in the app. A C call in 5.0.2 is an `OP_CALL` carrying a `CallInfo` and two `luaL_checknumber` conversions, where `if d < 0 then d = -d end` is three VM ops. The inline form is bit-identical to `fabs` here: the only value the two disagree on is -0.0, which the inline form leaves negative, and -0.0 compares the same against a positive tolerance and sums the same into a total starting at +0.0.

snaptol_firm = 1.25 %

How close the measurement has to land for a snap to be presented as certain rather than merely chosen. Inside `snaptol` the standard rate is still the reported answer, real devices using standard rates; past this inner band the measurement and the label disagree by more than the fit's own accuracy.

Calibrated, not picked. On one side sits the behaviour `docs/REFERENCE.md` documents and wants -- **29 127 Bd reading as 28800, 1.14 % away**. On the other sit the three measured cluster-C cases, whose raw fits land **1.63 %, 1.81 % and 1.97 %** from the standard rate they were relabelled as. A band at 1.25 % keeps the first confident and makes all three of the second approximate.

`snapped` and `firm` are separate returns on purpose. `snapped` is not a display flag: `ua_best` reads it as evidence when protecting the fitted rate, arming the submultiple guards and admitting alternative candidates, so narrowing it would change which bytes come back. `firm` is read only where confidence is claimed -- the panel's approximate marker and the rate auto-lock commits to.

baud_round cascades through three steps

The coarsest step that moves a measured rate less than 1 %: nearest 100 Bd over 10 kBd, else nearest 10, else nearest whole baud. A single fixed step made tidiness a **capability test**: with 10 Bd the only option below 10 kBd, every rate more than 1 % off a 10 Bd boundary was refused -- measured, **160 of the 901 integer rates from 100 to 1000 Bd**, so 105 Bd could not be locked, every recording mode refused it, and the operator was back at the keypad the rounding exists to avoid. Whole-baud always fits above the floor, half a baud being under 1 % of anything over 50, so `credible` is now true for any rate the decoder would accept at all.

That makes `autolock`'s gate 1 vacuous, deliberately. Credibility moves from "does this land on a tidy boundary" to "is this a rate at all", and the four gates after it carry the whole judgement: longest clean run over 60 % of frames, at least 8 good frames, fitq over 0.9, and a stable baseline. Those are about evidence; a boundary is about presentation, and presentation was never a reason to refuse a lock.

A rate already on the ladder is returned untouched, and the guard is not cosmetic: 31250 is on it and the 100 Bd step would move it to 31300, a 0.16 % shift that passes the tolerance. That is MIDI's rate, the one figure in the app that comes from a specification rather than a measurement.

Per-capture state

A measurement must not cross a capture boundary

Rate, format and polarity are derived from the signal, and a value carried across a capture boundary turns that derivation into a feedback loop -- the app's own previous answer becomes an input to the next one. Measured on the instrument on a 19200 line: a **leftover 500 kS/s makes a 19200 bit 26.04 samples, the width fit lands on 8.679 -- exactly one third -- 57600 is reported with nbad 0, and `pick_fs(57600)` writes 500 kS/s back for next time**. It is a two-state flip-flop rather than a one-way latch, so clearing once is not enough; it has to be every time.

`clear_measured` runs per **acquisition**, not per press, and the difference is four acquisitions wide: `autoset` runs the three-step probe ladder, then the real capture, then an octave retry, so a press that cleared only at its start would have its later looks reading the earlier looks' measurements.

Three things are deliberately kept across captures: the lock (`force_*`, dropped through `unlock_all`), `lvl_thr` / `lvl_hyst` / `lvl_swing` (the recording watchdog's fallback threshold -- under flow control the device is silent until the credit pulse goes out, so the probe that normally supplies a threshold looks at an idle line), and `flog_*`, the byte log.

fs_want: the second pass of autoset

`autoset` captures once at the top rate to find the baud, then sets `sdec.fs` to the lowest rate still giving 8 samples/bit and captures again -- the step that turns a 19-byte window into a 240-byte one. But `acquire` calls `hw_config`, which calls `fs_select`, and that only ever consulted `force_baud`: with nothing locked it reset `sdec.fs` straight back to `fs_default`. So the second pass ran at 1 MS/s exactly like the first, and an unlocked capture could never return more than ~19 bytes at 9600 baud however long the line talked.

Found on the instrument by pressing Options > Auto Detect and watching a **227-byte capture become an 18-byte one**. It was not the button: nothing unlocked had ever produced a long window, and the auto-detect path is simply where that is most visible. The field is one-shot, cleared on read, because it describes a decision made for the next capture only -- leaving it set would be a second kind of lock with none of the lock's UI.

sdec.tick: what a soak cell spends inside acquire

The bench's panel spinner can only be advanced from code that is running, and the longest thing a soak cell does is `sdec.acquire()` -- **3.4 s of a 5.8 s cell**, of which `acq_copy` is ~**0.455 s per capture** and there are about five captures a cell. Nothing in `bench/` executes during any of it, so a spinner driven only from the bench freezes for seconds at exactly the point where a wedged instrument would hang.

The hook is chunked rather than tested per sample: `math.mod(i, every)` inside the loop puts a modulo on all 20 000 iterations of the hottest loop in the acquisition path. Slicing the range leaves the inner loop byte for byte what it was, and with `sdec.tick` nil -- always, in the shipped app -- the only cost is one nil comparison per chunk.

The trigger model and the cancel key

arm_settle_s = 10

How long to wait for an armed model to reach its wait block. Measured on 1.7.17a from inside the instrument, 30 arms at each of two capacities, polling with `delay(0.001)`:

capacity	min	median	p95	max
21 100	0.1668	0.1917	0.1932	0.2032
860 000	0.0210	0.1928	0.1952	0.1973

The worst case of all, **0.228 s**, comes from arming right after aborting a model that was itself sitting in STATE_WAITING. So an 0.25 s budget stood about ten per cent above the slowest arm the instrument produces, which is why one hw-arm case armed once and failed the next three runs.

It does not scale with capacity. The arrival is flat from 21 100 to 2 800 000 readings, a 133-fold range, and five separate controls are nulls: `dmm.digitize.count` (1 against 817 000), a host `buf.clear()` before `initiate()`, samplerate (40 kS/s against 1 MS/s), template position (1 against 50), and the poll cadence itself (10 ms, 1 ms and 0.2 ms all alike). What moves it is what the instrument was doing before the arm. `buffer.make()` is not in this budget and is cheap anyway, **2.5 to 3.7 ms at 860 000 readings over 21 observations**; it runs in `acq_make_buffer`, before `stream_arm` is called.

A large bound costs nothing on a successful arm, the spin breaking on the first WAITING it sees, about nineteen looks. It is spent only when the model never reaches its wait at all, a firmware fault after which the recording runs free and still returns a result. Ten seconds is forty-four times the slowest arm measured. The

failure **inverts** the feature rather than degrading it, which is what makes a missable budget worse than a generous one: `not saw` is carried out as an arm that never went live, so the recording runs free and the operator gets exactly the buffer of silence that pressing Capture on a quiet line avoids.

buf.n is not evidence of a trigger

An armed recording's poll loop has to distinguish "the device has not started yet" from "it started and the burst is running", and it cannot do that from the buffer: the pre-trigger phase accumulates readings while it waits. Measured -- an 8 kB recording armed on a silent line **plateaued a few readings above its reserve within milliseconds of `initiate()`**, which a buffer-level test read as "fired", and the idle watchdog then ended the run as 'quiet' one second later with zero bytes, three seconds before the device started transmitting.

`trig_waiting` answers true, false or nil, and nil means degrade rather than retry: a state read that raised leaves the caller with no way to bound an arm, and the honest answer is to behave exactly as a recording did before the feature existed.

The front TRIGGER key is the only stop control this firmware allows

For a job longer than one press. A touch press is not dispatched while Lua runs -- presses queue -- so no button on the glass can stop a loop that has not returned; `display.waitevent()` blocks and wedges the instrument; and `trigger.wait()` reads *TRG out of the command queue rather than watching the panel (ref 14-417). The trigger subsystem works because it is firmware and needs no interpreter: the key generates `trigger.EVENT_DISPLAY` and an event blender latches it.

Measured on 1.7.17a by `tools/bench_cancelkey.py`: a press latches with nothing armed; the latch is set **while Lua spins** (pressed during a deliberate 10 s busy loop, seen by the poll afterwards, twice); `wait(0.001)` returns in **1.1 ms** and `wait(0.05)` in 50.1 ms, so it polls rather than blocking; **1.06 ms a poll**; and nothing reaches the event log.

The timeout handed to `wait()` is never zero: `display.waitevent()` reads a zero timeout as "wait for ever", and wedging the instrument that way cost a power cycle. 1 ms is the smallest value measured to return promptly, and 0.2 % of one decode window.

A bus trigger is deliberately not a second stimulus. `trigger.EVENT_COMMAND` (*TRG) looks like a free way for a harness to cancel a run without a finger, and it does not work: measured on 1.7.17a, a *TRG sent 2 s into a 6 s busy loop **did not reach the latch**, the command queue being drained only once the interpreter is idle. A stimulus that can only fire between runs -- where `cancel_clear` discards it -- would be wiring that reads as coverage and is not. What does fire mid-script is a trigger **timer**, which is firmware: `tools/bench_panel.py` adds one for the duration of the cancel case and removes it afterwards.

The armed wait must outlast the capture itself

Completion is the model going idle, which needs all n post-trigger readings: **20 000 at 2500 S/s is 8 s against a 3 s wait**. A fixed wait therefore times out on every capture below ~7 kS/s and degrades to free-run with 'edge trigger unavailable' -- a working analog trigger reported as absent. Measured at 300 baud.

A bare `waitcomplete()` is a hang: `LoopUntilEvent` makes readings until it detects the event, so on a line with no start bit -- a dead or grounded pin, which the probe treats as the normal starting condition -- the model never leaves its wait block and Lua never returns from inside a touch handler.

A completed armed capture holds 19 011 of 20 000 samples and then goes STATE_IDLE. Never 20 000. Measured at 9600 baud into the front terminals. `position` is the pre-trigger percentage and pre-trigger readings accumulate only until the trigger fires, which on a continuously transmitting line is within

about fifteen samples rather than the 1000 that 5 % of the buffer allows for. So a test of `buf.n >= n` cannot be satisfied by any capture that worked, and with it every edge-triggered capture times out, aborts and falls back to free run -- the analog trigger working the whole time, the model firing within 0.2 s, and none of it showing.

Fill level is still polled, but only as an early exit for pretrig = 0, where a full buffer is the completion condition. It cannot stand alone: on a quiet line the model keeps making pre-trigger readings and wraps the buffer, so `buf.n` reaches capacity with no trigger ever having occurred, and pre-trigger samples of unrelated activity decode perfectly well and would be presented as "captured on a start bit".

Arm Wait applies only where the comparator is the source

The 120.05 s figure below is the defect two guards removed, not current behaviour. It is recorded because it is why the guards exist.

Measured before either guard: with Arm Wait at the form's 120 s ceiling, a 'front' press on a quiet line **froze the panel for 120.05 s**, and the expiry message deliberately refuses to name Arm Wait for that source -- so the operator had a two-minute freeze and nothing linking it to the field they changed. The same freeze was reachable a second way, by swallowing `arm_comparator`'s verdict: that loaded the model on an event nothing was configured to raise, then offered advice about two settings that cannot help at any value.

Both routes are closed. The comparator verdict now raises rather than loading an unraisable model -- `error('no analog comparator on this digitizer', 0)` in `sdec.acq_triggered()` -- and the armed wait is polled from inside via `sdec.arm_wait_s()` in `sdec.stream_acquire()` instead of blocking on `trigger.model`, so the panel stays live for the whole wait. Verified in the code; the absence of the freeze has not been re-measured on hardware.

A nil event constant reaching `trigger.model.load()` takes it **out of band**: the model loads, waits on nothing, and the panel freezes for the whole of Arm Wait against working hardware. `trigger.EVENT_ANALOGTRIGGER` is absent on a digitizer with no analog comparator, which this app treats as a supported configuration, and `EVENT_EXTERNAL` need not exist on every model; neither is nil-tested anywhere else in `tsp/`.

The flow-control credit pulse on the free-run path

An armed capture degrades silently to free-run when the trigger source is refused, so a credit issued only on the armed path can never fire -- and flow control is the only thing preventing byte loss between windows above 4800 baud. Measured on scope CH4: `assert()` gives **4.92 V / 5.88 V pk-pk**.

The credit goes out **after** `initiate()` and before the read. `trigger.model.initiate()` arms the digitizer and returns immediately, the model running in firmware, so by then the instrument is already listening; asserting before it would let the head of every frame arrive while nothing was armed, losing exactly the bytes the feature exists to save on every frame. On the free-run path `dmm.digitize.read()` blocks until it has n samples, so a credit issued after it invites the device to transmit into a window that has already closed.

hw_verify reads back three attributes, once a launch

`hw_config` writes four attributes and read back none, so an instrument that cannot honour a value snapped to a neighbour and said nothing. **The 2461 defaults its digitize voltage range to 7 V** where both DMMs default to the 10 V `sdec.range` pins, so that is the model this catches. Three and not four: `acq_measure_fs` re-derives the true rate from the buffer's own `relativetimestamps`, a stronger check

than a readback. Once a launch, nothing inside a load changing these three, and they are proxy reads in the phase that already dominates cell time. A mismatch is not an error -- the capture is valid, just coarser than asked for -- so it goes to the log and to `sdec.hw_snap`, never to `stickyerr`.

Detection from a prior

A failed re-capture leaves a stale fit

`autoset()` runs up to three captures per press, and a pass that fails at `acquire()` returns before any decode. So the previous pass's `res`, `baud`, `bittime` and `fitq` survive beside the failed capture's `acq_fs` and levels, and the panel reads a rate measured from samples it no longer holds.

`drop_stale_result()` exists for exactly that, and it is deliberately not `clear_result()`: `clear_result()` also drops `probe_note` and `lasterr`, which on this path are the only record of what failed. It is called at every early return past the first acquisition -- the probe's own failure, both second-pass failures, and the octave retry.

The values it writes are `clear_result()`'s, not nil: `ui_field_colour` reads `fitq` numerically, so a nil there colours a field from a comparison against nil.

The prior ladder needs the pulse-width filter

`arm_try_ok()` is the filter the prior ladder applies before spending a decode, and it is the detector's own three tests reused: `ua_plausible`, `ua_submultiple` and `ua_minrun_absurd`. Without them the ladder tries every rate in `sdec.arm_try` and the frame gate cannot rescue it, because a wrong bit time still frames bytes -- half the true rate reads two real frames as one and lands its stop bit where a real one is, which is the failure 'Why the rescaling search is gated' and 'ua_submultiple: the odd-multiple test' describe from the other side.

It fails closed with no evidence. All three tests answer "no objection" to a missing width array, so the function tests `wmed` and `nw` itself and refuses; `sig_bittime()` must have run first. `arm_fit_prior()` calls `sig_bittime()` on its own samples rather than inheriting whatever ran last, because the previous capture's median pulse rejected the true rate of this one.

arm_badfrac = 0.02

How clean a guessed rate has to decode, as a fraction of interior frames in error. Far tighter than `relock_badfrac` = 0.25 and not interchangeable with it: `relock_badfrac` asks whether a rate the operator typed has been discredited, `arm_badfrac` asks whether the app may invent one.

The two sit either side of the measured separation in 'relock_badfrac = 0.25': a wrong rate runs **0.34-0.50** interior-bad and a healthy capture **~0.004**. 0.02 is five times the healthy figure, so jitter does not trip it, and an order of magnitude below the wrong-rate band, so nothing in that band can be published as an invention. The frame floor beneath it is `relock_minframes`.

Uniqueness, not first fit, for a guessed rate

Clean frames at a wrong rate are the normal outcome, not the exception, so the first candidate that frames cleanly is not the answer. `arm_fit_prior()` decodes **every** surviving candidate and returns true only when exactly one of them is clean. That costs CPU and nothing else -- no byte of the device's start is traded for the attempt, because the attempt runs over samples already in hand.

Two or more clean fits is itself an answer worth publishing. `probe_note` then names the two rates the capture cannot tell apart, which is what lets the operator resolve it from the one thing the instrument does not know, and the function returns false rather than guessing between them.

The ladder is `sdec.arm_try` = {9600, 19200, 28800, 31250, 38400} (serial_core.tsp): a prior, not a measurement, with the two commonest bench rates first. Nothing below 9600, because a slow line gives the detector hundreds of samples a bit and never reaches this fallback.

The loop and the final re-decode are each wrapped in a `pcall`, and the restore of `sdec.force_baud` sits after both. `decode_from()` can raise, and a raise past an unrestored `force_baud` leaves the operator's Auto silently converted into a hard lock. The wrap is around the whole loop rather than each candidate, because a raise defeats every hypothesis equally.

It calls `decode_from()`, not `decode()`: `decode()` second-guesses a forced rate through the relock path, and here the lock is this function's own hypothesis and the fallback measurement has already failed.

The probe pass

Event 4915 on the probe pass

The probe pass free-runs whatever trigger mode the operator selected, and `sdec.trigmode` is forced to 'free' for its duration. Arming a trigger model here posts event **4915 ten times a capture at ERROR severity**, which `showevents` cannot mute, so the event has to not happen rather than be filtered.

4915 is a buffer-allocation failure and raises no dialog, which is why it can fire ten times a capture unnoticed; **-109 is the positive control** that proves a probe can see one at all. See 'Event 4915 raises no dialog'.

The cost of free-running is that a 20 ms window can land inside a slow line's idle gap, and the retry ladder is what pays it. The restore goes through `pcall` and is also registered as `sdec.trig_restore`, because `sdec.guard` can see a field where `savetrig` is a local it cannot: a raise inside `acquire()` would otherwise leave `trigmode` on 'free' for the rest of the session while the panel still reads Edge.

The probe ladder and the octave retry

Two passes, because sample rate trades resolution against duration and only one of the two is knowable in advance. 1 MS/s oversamples anything up to ~125 kBd but fills the 20 000-sample buffer in **20 ms, barely 19 bytes at 9600 baud**. So the first pass finds the baud rate at the top rate and the second re-captures at the lowest rate still giving 8 samples/bit. A forced rate skips the first pass.

Each retry looks for **longer**, by dropping the sample rate rather than repeating the same window. `sdec.probe_fs` = {1000000, 100000, 10000} is 20 ms, 200 ms and 2 s for the same 20 000 samples, and 20 ms is less than one frame at 300 baud. The first look keeps the resolution a 250 kBd line needs.

A probe that measured the bit time but framed no byte is not a failure: 20 ms at 1 MS/s is six bit times at 300 baud. The retracted rate in `sdec.baud_probe` is kept for exactly this -- it chooses the second pass's sample rate and is never displayed. See 'The retracted bit time is kept for autoset'.

The probe can also be an **octave out**, and it costs half the window when it is: every edge interval that is a multiple of T is also a multiple of T/2, and the probe has only 4 ms of signal to separate them. The retry fires once and only downward -- the guard is `w2 < sdec.fs`, so a retry can only lengthen the window and increase the byte count, and cannot oscillate.

Arming from silence needed a locked rate to reach

A line that has not started is not a probe failure; it is the armed case. The device transmits when it is powered on, and until then there is nothing to measure, so `autoset()` falls through to `arm_capture()` when the probe found no traffic. Before that fall-through existed, an armed capture was reachable only with a rate already locked, which is the one state a first-contact operator does not have.

Two conditions gate it. `sdec.arm_silent()` (serial_core.tsp) is the predicate, and it is a function rather than a read of `sdec.probe_idle` because `probe_idle` means "the probe saw no transitions" -- equally true of every flow-controlled capture, where the probe runs nofc and the device has had no credit yet. So `arm_silent()` is `probe_idle` **and not** `fc_out`.

`savetrig ~= 'free'` is the second: arming is still refused where there is nothing to wait for. A free-running capture has no trigger to arm, so an armed retry there would block for the wait length and return the same silence.

Flow control and recording

fc_maxwin = 32 and fc_maxsec = 1200

What one press may spend under flow control. The normal ending needs no bound at all -- a credited window that comes back quiet means the device has finished -- so both of these are for the device that never stops.

Two bounds, because a window count is not a time: each window's acquisition is bounded separately and the decode costs more than the acquisition did. **fc_maxsec is the one that binds.** 1200 s is where `stream_maxsec` already caps, and two bounds that disagree would make the smaller a hidden cap; for scale, a 32 kB window is **1092 s at 300 baud**.

Under flow control the ceiling costs nothing but another press, because the sender is still waiting where the run left off. These bounds are the only exit: `cancel_pressed()` answers false for ever while a panel-initiated run executes, since a press on TRIGGER emits no blender event then, so nothing else ends the loop.

A reused threshold can sit on a rail

The recording watchdog re-measures levels mid-run and decides whether to adopt the new threshold. Wrong means the signal cannot cross the old one, not merely that a new measurement differs: a threshold still between the two levels **with room to spare** is doing its job, and adopting a marginally different one on every window would restart the watchdog for ever.

"Between the levels" is not "crossable". A bare `lo < thr < hi` admits a threshold sitting **on** a rail, where the signal reaches it but never passes it and the edge list empties. The margin is the hysteresis, which is already the app's definition of what counts as a crossing, and it is taken from the levels just measured because it is the current signal that has to cross it. With no hysteresis available the fallback band is **10 % of the measured swing**.

The test is `oldthr > lo + band and oldthr < hi - band`; failing it is what adopts the new threshold, and the adoption is cleared so it runs once per recording.

Reading memory

ck_bufwant = 4076503

Readings the largest recording may ask for. The 32 kB mode at 9600 baud records **3 413 334 readings at 100 kS/s**, so any ceiling above that keeps all 32 768 bytes; the figure is self-checking -- 32 768 bytes x 10 bits / 9600 baud is 34.13 s, and 34.13 s at 100 kS/s is 3.41 M readings. What 4076503 buys over 3.41 M is every other reachable mode-and-rate combination.

Bisected against the instrument, not chosen: it is the largest request the meter granted once the default buffers were reclaimed, so the last digits carry the allocator's own granularity and are not meaningful on their own.

`ck_bufmax` is the live ceiling `start()` recomputes from the capacities the operator has actually set, initialised to `ck_bufwant` so a caller that never ran `start()` still has a number. `bufmax_for()` reads `ck_bufwant` and never `ck_bufmax`, so a launch under a large `defbuffer1` cannot ratchet the static need downward.

The reading pool is per model

The instrument's whole reading memory, across all buffers together, **standard style only**, which is what `acq_make_buffer()` allocates.

model	STYLE_STANDARD readings	source
DMM6500	**7 080 255**	the shipped default; `docs/UNDOCUMENTED.md`, 'The real ceiling is 7,080,255 readings, across all buffers together'
DMM7510	7 500 000	reference 10702
2461	6 875 000	reference 7716

The 20 M and 27.5 M figures that appear in the vendor material are **COMPACT**, which is not a reachable ceiling here: see 'buffer.STYLE_COMPACT is nil on this firmware'. Quoting them would overstate the pool by millions. Neither figure appears anywhere in `tsp/` or in this doc as a limit.

Standard style is also right on its own merits, because payload size costs time where a reading is **written** and is invisible where it is read. Acquiring 20 000 samples is **207.96 +-0.90 ms** standard against **221.65 +-2.17 ms** full (+6.58 %, 8.3 sd), while a proxy read of one reading is **22.820 +-0.013 us** against **22.836 +-0.016 us** (+0.07 %, 1.1 sd -- nothing).

`buf_pool_resolve()` is a `localnode.model` lookup and not capability detection, deliberately: nothing can answer a pool size at runtime without failing an allocation, and a failed allocation posts 4915. An unlisted model keeps the shipped figure, so the lookup refuses nothing it cannot explain. The default lives in its own field (`ck_bufpool_default`) because a resolver that assigns only on a match leaves the previous match in place; this one assigns unconditionally from a local, so the answer depends on the model and not on call order.

The window that made the per-model table necessary: on a 7510, any `defbuffer1` + `defbuffer2` total in **(2 953 752, 3 423 497]** had the app resize the operator's buffers when the meter in fact had the room.

Beneath the pool sit two separate numbers. `ck_bufguard` = 6 000 000 is the hard guard and is what the vendor's `bufferVar.capacity` documents. `ck_bufmargin` = 50 000 is a chosen margin, not a second measurement: it covers the pool figure's uncertain last digits and the firmware's unreported per-buffer

overhead, and at **0.7 % of the pool** it costs nothing.

ck_bufmin = 898347

The smallest recording that is still a recording, and the floor `start()` refuses below. Under it every Capture press would end in the allocation refusal, which puts the failure three screens into the app instead of at launch where the operator can act on it.

The refusal names what it tried rather than just the shortfall -- the readings left, the readings the smallest mode needs, which default buffer holds how much of the pool, and whether resizing them back was refused or merely did not free enough -- because on the measured pool this branch is reachable only where the reclaim could not help, meaning a meter that refused the writes or will not report its sizes. Reaching it also hands the capacities back, since that launch is not happening.

Derived, not measured. It is the 8 kB mode's own capacity under the rule `sdec.acq_cap()` states and `sdec.stream_begin()` re-applies, with `sdec.pretrig` = 5:

```
nsmp = ceil(8192 x 10 x 100000 / 9600)       =   853 334
cap  = ceil(nsmp x 100 / (100 − 5)) + 100    =   898 347
```

Both lines reproduce the constant exactly. The order matters: the **+100 overshoot headroom lands after the ceil**, and applying the reserve and the headroom the other way round gives 898 352. That is why the obvious hand arithmetic does not close -- 8192 bytes at 9600 baud sampled at 100 kS/s is 853 333 readings, and neither the pre-trigger reserve nor the headroom is in it.

The same formula gives the 32 kB mode **3 593 084**, which appears verbatim in the commit message of `2b4fe67`. That is the independent confirmation, and it is also where the derivation lives: no comment in the module records it. `git log −S'898347' −−all −− tsp/` finds it in one command.

Note what the figure is scoped to. The reserve term is reached only when `sdec.trigmode ~= 'free'` and `pretrig > 0`, so 898 347 is the **triggered** floor; on the free-run path the same mode needs `nsmp + 100` = 853 434. `ck_bufmin` is the triggered figure because the probe that decides silence reads into that buffer too.

Buffer capacities across a relaunch

A relaunch is not an exit, so the reclaimed default buffers stay reclaimed across it. `ck_bufmax` was derived earlier in the same `start()` from the reclaimed sizes, so growing `defbuffer1` and `defbuffer2` back underneath that ceiling would leave every later decision measured against memory that is not there. `sdec.ck_relaunching` is the flag that suppresses the hand-back, set around the `cleanup()` call and cleared immediately after.

What makes a rebuild safe at all is the teardown. A second `ui_build()` over live objects crashes the firmware hard enough to need the front-panel POWER button, which costs a reboot and a reinstall, and remote recovery is thin: POWER is a soft button and the LXI panel states that the instrument cannot be switched on or off with the power switch. `cleanup()` deletes every object it created, nils the handles and collects, so nothing survives to be created over.

`delfails` counts refused deletions, and a non-zero count -- and only that -- refuses the rebuild, because it means objects have leaked. `del_orphans()` runs first, since anything a previous teardown could not delete is reclaimable and should not be counted as a leak. `sdec.allow_rebuild` overrides the refusal and is opt-in.

The launch capture and the pcall budget

On firmware 1.7.17a a capture taken from inside `start()` bluescreens the instrument when the app runs from **internal memory**, at **PC 001CF3C8**. The cause is `pcall` **depth**: that path sits one layer below a firmware stack limit, and what matters is the **peak** depth reached anywhere during the capture, not the depth of the chain on the way in. Flattening the chain did not fix it; reducing the app's `pcall` count did.

So the budget is a shipping constraint with an unknown margin.

count	outcome
191	bluescreen
132	measured clean
113	the tree now

The boundary is somewhere between 132 and 191, and nothing narrows it further. The count is of `pcall` **source sites** across `tsp/`, so comment lines have to come out of it first -- **four** comments mention `pcall(`, and the one-liner that forgets them reports **117**:

```
grep -h '' tsp/*.tsp | grep -v '^[[:space:]]*--' | grep -c 'pcall *('
```

Two traps in that command. `grep -c` counts **lines**, not occurrences, so a line carrying two `pcall(` would count once; no line in `tsp/` does today, which is why 117 is both the line count and the occurrence count, but the next such line makes the two diverge silently. And `\s` is not a shorthand class in a BSD basic regex, so `'\bpcall\s*('` does not mean there what it means under GNU grep -- use `' *'` or `'[[:space:]]*'`.

Current distribution: serial_app 47, serial_core 30, serial_ui 28, chunk_decode 6, usb_log 2, and none at all in uart_decode, lin_decode or midi_decode.

Guards belong at a **firmware boundary** and never around a call into our own code, which is what the count is made of; `tools/harden_formats.py` exists because removing a guard exposes any nil it was hiding. **No size threshold explains the crash**, so making the app smaller buys no headroom.

`start()` therefore returns with the digitizer untouched and `sdec.res` nil: the app comes up idle and the first acquisition is the operator's first press.

Auto-lock

Auto-lock pays for itself

An unknown format costs the format search -- **twelve decodes** against framing a known one once (2 widths x 3 parities x 2 polarities) -- and `autoset()` spends a whole extra capture at the top sample rate purely to find the baud rate. Locking removes both, so **the second press onward runs about three times faster for the same bytes**.

It also buys something the operator cannot get otherwise: the streaming modes require a locked rate, so until something locks they refuse outright.

The gates are the whole design, because locking a **wrong** detection turns a one-capture mistake into a persistent one. The five in `autolock_try()` are independent -- timing, framing, sample size, fit quality, baseline stability -- and five correlated checks would be one check.

`autolock` is operator state, so it is nil-tested rather than assigned: someone who turned it off must not find it back on after an App relaunch. It is on by default, so the fast streaming-capable state is reached without being asked for. `autolock_defer()` suppresses it for the next capture only, because an explicit request for auto-detect must survive the capture that follows it -- otherwise the same press re-locks the rate just detected -- and one capture of grace is the whole of it, the request having been "detect it", not "never lock again".

autolock_run = 0.6

The fraction of the capture that must be one unbroken correctly-framed run before a detection may be locked. Well short of 1.0 because the auto-detect capture is the shortest the app takes: at 1 MS/s the 20 000-sample window holds about 19 bytes at 9600 baud, and `ua_edge_frames` already excludes three frames at the head and one at the tail as capture-boundary artefacts.

It sits alongside `autolock_min` = 8 frames and `autolock_fitq` = 0.9.

What gets locked is the **rounded** rate, so a non-standard measurement locks as a number an operator could have typed -- 16099 locks as 16100. `sdec.baud` is then republished as the locked value with `snapped` and `snapfirm` both true, because `baud_text()` marks an unsnapped rate with '?' and a rounded 16100 beside a `sdec.baud` of 16099 drew **'16099?' under a green padlock**. `baud_raw` is left alone: it is the honest measurement, and the forced-rate cross-check compares against it.

The note names the payoff rather than the fact -- the next press's sample rate and window size in bytes, the window being the number the operator cares about -- and is sized to the note cell at **685 px against the 700 of `sdec.ui_note_px`**.

Polarity is not locked

The one asymmetry in the locked set, and deliberate. Left nil, `decode_from()` falls back to the level-based inference from `sig_idle` and skips the polarity contest entirely. Locked, it would freeze whatever that contest decided on the shortest, least evidenced capture the app takes -- and 'polmargin = 4, and the inverted-ASCII trap' is the measurement of how wrong that contest can be on one short window, where the inverted reading of ASCII text returned 18 frames with zero errors.

Rate and frame shape are worth locking because re-deriving them costs a capture. Polarity costs one test, so there is nothing to buy and a wrong answer to avoid.

The panel

The panel coordinate space

An object's y is relative to the screen CONTENT area, **49 px** below the panel top, so a child at y = 350 lands on panel row 399; x needs no correction. The content area is therefore 480 - 49 = **431 px** tall. Coordinate limits are **X 1..798, Y 0..420**, and the firmware accepts a y past the usable bottom without complaint, clipping the object instead -- `tools/mock_display.lua` raises on it so an over-reach fails a test rather than a panel that cannot be rebuilt without a power cycle.

`display.create` refuses a width of 800 (error 1130) and **returns nil rather than raising**, so the object silently does not exist. A screen title over **31 characters** is truncated (error 1707).

The firmware inks exactly w columns from x, measured on a grab: the header's horizontal rules run 1..796 and the 1 px vertical at 797 inks column 797 alone, so 798 and 799 stay dark. `mockup.lua` reads a rect's extent as x + w, so at x1 = 798 it reports the right-hand vertical as reaching 799 and refuses it as off-

screen -- hence `ui_rule_x1 = 797`. The render's arithmetic is one pixel more generous than the instrument's; do not reclaim those columns on the render's word.

A **rect spans y..y+h-1 and x..x+w-1**: the status divider declared at (472, 330, 1, 19) inks object rows 330..348, and row 349 at that column belongs to the bottom rule, which is what makes it look 20 rows tall.

A text object's y is the BOTTOM of its glyph box and the object grows upward, so its ink band is [y - ink + 1, y]. Measured from a panel screenshot: dump rows created at content y = 66 ink panel rows 104..115, i.e. content 55..66.

Glyph ink heights, measured off `panel-v3.png` in regions where only one row's ink could fall: FONT_SMALL **12 px**, FONT_MEDIUM **17**. A medium value at y = 28 inks 12..28 while a small label at y = 14 inks 3..14, so those two collide over three rows. `ui_ink` measures the glyph box and **excludes descenders**: on the note row's lower-case prose the deepest tip is at 55 against a nominal 51, and the status row's at 347 against 344.

A button is **58 px tall** on the main screen (`docs/panel-ref-chrome.png`) and **60** on the options screen (`docs/img/options.png`); `display.create` takes (x, y, text, width) with no height parameter, so a button cannot be made shorter to find vertical space. 58 px needs y <= 373.

Character advances on the three panel faces

The face is proportional and **uppercase is ~25 % wider than everything else**, so a label's width does not follow from its length:

run	px/char
hex digits and spaces	7.47
mixed-case ASCII (the dump's gutter)	7.73, 7.80 worst case
UPPERCASE (every header label)	9.0 .. 9.67
FONT_MEDIUM digits	~13.4, ~14 worst case
button faces	12.7 .. 15.5

`ui_adv_upper = 10` and `ui_adv_mixed = 8` are those worst cases rounded up. FONT_MEDIUM's 13.4 comes from a panel grab where an 8-character LOGIC value spanned exactly its 107 px column.

A button face needs len * 14 + 8 px to render its label uncut. Calibration points: 'Save USB' is clipped at a width of 112, 'Lock Detected' at 150, and 'Options' (7 chars) just fits at 112.

`ui_ch_w = 8` is **not** one of these: it is an estimate of the dump rows' advance, Menlo at 14 px, which is what `tools/render_png.py` uses to approximate FONT_SMALL. The real face is unmeasured, so every dump column is computed from it -- at 8 px/char the hex block is 376 px wide and at 11 it is 517, so a hardcoded ASCII column would sit 141 px inside the hex it follows.

ui_cw, and the 1.159 render-to-panel factor

`ui_cw` holds a width for each of bytes 32..126 as px + 48, so the table is a printable literal. There is no text-measurement call on this firmware and the face runs **3 px (an 'i') to 17 px (an '@')**, so a character count safe for a wide string wastes a third of a cell on a narrow one.

The widths are the mockup's **scaled by 1.159**, measured off a screen grab: the 74-character 250 kBd warning draws **568 px on the panel and 490 px in the mockup**, so `render_png.py` 's face is ~16 % narrow and calibrating against it would under-measure every string.

`ui_fit_slack = 1.04` is the headroom every `ui_fit` budget keeps, because `ui_cw` is a per-character model against a proportional face and so reads narrow: measured, the 104-character mid-byte note fitted to 700 px models at 696 and **renders 706**, 1.4 % over. From x = 92 that puts its tail past the 798 px edge, where the firmware cuts a string without a mark, and the same 1.4 % is enough to overwrite the 'p' of the page counter 8 px away. 1.04 rather than 1.014 because the error is a bound and not a constant; 4 % of the note cell is 28 px, three characters of a sentence that is being ellipsised anyway.

Where the header table rules go

Every vertical position is chosen so that no horizontal rule lands inside any glyph band, and `tools/test_serial.lua` asserts exactly that -- a rule inside a band draws a line struck through the text, and the one-build-per-power-cycle rule means a layout error costs a power cycle to see.

rule	row	clearance
`ui_rule_hdr`	15	between the label ink (3..14) and the value ink (16..32), one free row
`ui_rule_mid`	37	the `fd.dy` cells' medium ink reaches 34; at 35 the rule read as underlining them
`ui_rule_bot`	56	one clear row under the note row's descender tip at 55
`ui_stat_top`	330	the dump's last row inks to 320
`ui_stat_bot`	349	one clear row under the status row's descender tip at 347
`ui_btn_y`	362	58 px of button inside a 431 px content area

The two headline cells carry `fd.dy = 2`, and FONT_MEDIUM inks 17 px against FONT_SMALL's 12, so they reach **5 px higher** than the nine beside them: their band top is 17 where the small cells' is 21, against a header rule at 15. A check written against the small band alone leaves the tallest cells -- the only ones that can reach the rule -- unguarded. `sdec.ui_field_band()` is the one function both the panel and the test derive a band from.

One px of `fd.dy` is also what makes the row read as level: drawing both rows off one baseline sits the two big values a pixel high against the nine small ones.

The header column widths

Each column is sized for the **widest** value it must hold rather than a typical one: BAUD holds six medium digits (115200, 250000) = 84 px, so its column runs 8..115. A collision that appears only when someone probes a fast line is the kind that ships. BYTES holds six digits because the streaming modes report 32768 and up, ~48 px.

The padlock sits at x = 95, after the widest value BAUD can hold rather than the one on screen: '115200' is 6 x 14 = 84 px from x = 8, ending at 92. The BAUD column is 107 px to hold value + gap + glyph.

Every cell but the two flagged `med` is FONT_SMALL, because each field x is sized on the 8 px FONT_SMALL advance that also governs the dump columns, the vertical rules and the collision check in `render_png.py`. A FONT_MEDIUM value advances ~13.4 px, so an 8-character LOGIC value ('3V3 CMOS') takes 107 px and touches THRESH.

`fd.labdx` nudges the label and never the value: labels sit at a uniform 9 px inset from their cell's left rule, which reads as a grid everywhere the label is narrower than its column. FORMAT is 63 px in a 72 px cell, where that inset butts the word against the divider.

The progress bar extents

Every extent is derived from the three rules that bound the status cell, so moving any of them moves the bar. Written as literals they go stale silently: the bar is invisible in frame mode, so a stale y draws a grey frame through a rule only while a recording runs.

`ui_prog_x = 339` is measured with `ui_textw` against the widest string `ck_status()` can return while running, the flow-controlled decode: '32 kB decoding... win 32/32 32768/32768', **311 px**, so it ends at 319 and 20 px of clear space puts the bar's left edge at 339. `test_serial` recomputes the 319 from the format strings themselves, so a longer message fails a test instead of being drawn through the bar.

Thickness caps at 10 -- 12 is refused with "1130 Parameter thickness, expected value from 1 to 10", which raises a modal dialog over the panel. That ceiling is why the trough is 2 px and not 1: the frame's interior is 14 rows, so one empty row above and below would need a 12 px stroke. The 4 spare rows are split evenly instead, an uneven trough reading as a bar sitting crookedly in its frame.

A stroke of thickness lw declared at row r inks r - floor(lw/2) upward and lw - 1 - floor(lw/2) downward, measured on a grab: lw = 10 at row 339 inks 334..343.

The bar is an OBJ_LINE and the frame an OBJ_RECT because of what this firmware draws: a rect is an **outline** `thickness` px wide in its setcolor colour, `display.setfill`'s percentage has no visible effect on one (a 64 x 14 rect at setfill(100) lights 137 px, its perimeter, not the 896 a filled one would), and setthickness caps at 10, so a rect cannot be thickened into a solid 62 px bar -- at thickness 6 it drew a 6 px border with a hollow middle. An OBJ_LINE at thickness 10 is solid at any length and `setposition` moves its endpoint, giving 1 px granularity with no ladder of segments.

The bar is created **before** `ui_page_t` because creation order is z-order here, and the resting status strings are longer than the running ones: '32768 bytes 999 err 9 win -> bytes009 [full]' reaches **404 px** past the bar's 383, so a rect created later would paint over the tail of the sentence naming the file.

The three status row cells

`ui_stat_div = 472` splits the row; `ui_log_px` is derived as `ui_trig_div - (ui_stat_div + 6)` = 666 - 478 = **188 px**. The divider sits at 472 rather than 462 because widening the offset field to five digits pushed both hex groups 8 px right.

The obvious log form does not fit: 'log: bytes007.txt 12345 B' measures **195 px** -- 7 px over already at five digits, 204 at six and 213 at a megabyte -- and the firmware fades an overlong string's tail rather than clipping it, so the number goes unreadable exactly as it gets interesting. One press files up to 32 kB, so four of them cross 100 000 bytes. **Dropping the '.txt' buys 24 px**, bringing five digits to 171 and six to 180; a seven-digit count is 189 even so, so the NAME is what gives way, trimmed from the left with a '~'.

The first trim step drops four characters, not one: '~' costs 10 px and the character it replaces may cost 3, so a one-character step can make the string wider -- measured at 163 px going to 170 on a name of 'i's. Four narrow characters are 12 px, which covers the '~' outright, and every later step already has it.

'log: NOT LOGGING' measures **156 px**, so it always fits and never needs the clip. `flog_why`'s own sentences do not: 'no free bytesNNN.txt name' is 208 px and 'cannot append to /usb1/bytes000.txt' 271, against this cell's 188 -- hence a flag here and the reason on the note row.

The rear BNC cell at x = 675 offers **122 px** and 'EXT TRIG OUT' is **123**, one px over, so its labels lost the 'EXT': 'TRIG OUT' is 84 px.

The left cell has ~**454 px** before the divider when nothing is running, and `ck_status`'s widest ready string measures **438**. A verbose paging form -- 'FRAME TEXT page 1/1 bytes 1-1200 of 1200 win 240 [done]' -- runs to 60-odd characters, which at 7.80 px/char is past the divider and through the log field, hence 'pg'

and 'N-M/T'.

The note row pixel budget

`ui_note_px` is derived as `ui_rule_x1 - ui_note_val_x` = 797 - 112 = **685 px**, and was the literal 700 until the trigger-source cell beside it took 20 px -- a literal would have let the note draw through its own left rule.

The trigger-source cell is the row's left cell, its vertical rule at `ui_note_val_x - 9` = 103, which the longest label clears by 8 px. 112 rather than 92 because the cell holds 'Trigger key' at 84 px where mode names topped out at 62. The panel-edge check in `render_png.py` catches only a string that overruns 798, not one that overruns an interior rule, so a cell bounded by its neighbour is measured by hand; `test_serial` measures every label in `ui_trigsrc`.

The page counter shares the row, ending 4 px inside the right rule at `ui_pgind_x = 788`. Its digit columns are sized for **two** digits and a wider count overflows left into the gap: sizing for 137 pages renders 'page 1 of 35', 24 px of slack around every ordinary number to hold a case almost nobody meets, where two digits lets three digits eat 8 px of the 6 px gap. `ui_note_gap` is 20 px and not 12: at 12 the note's own width model could overrun the 8 px left, and did -- the yellow note drew over the 'p' of 'page'.

The note has 556 px when the counter is up. The mid-byte note's long form measured 757 px and rendered as '...everythi...', which is what cut every trailing reassurance from it. `mode_why`'s sentence is the other way round: in full it measures **802 px against a 798 px panel**, so the status row carries 'not armed -- see the note' and the note cell carries the remedy.

Display objects: cost, census and the dump-row choice

A text object costs ~12 ms to create and one display message per refresh, and **~192 messages per refresh** is the flood that hangs the display processor. Display calls are asynchronous messages to the second PowerPC over a bounded, overrunnable queue -- "1706 Message dropped from display queue" precedes the firmware panics -- so rewriting unchanged rows is pure queue pressure, which is what `ui_set`'s shadow cache exists to avoid.

Object census, counted by `tools/mock_display.lua`: **122** objects live after `ui_build()` and **134** once the options screen is built too, 12 of them its own. Any reasoning about how close the pool is to exhaustion starts from those two numbers. At ~12 ms each that is well over a second of drawing, which is what the loading cover hides.

Objects per dump row, with each block at its own fixed x so no block can drift into the next:

layout	objects	note
4/row	60, <= 60 writes/refresh	shipped: offset, two 8-byte hex groups, ASCII
7/row	105	offset + four 4-byte groups + ASCII; bounds drift to 4 bytes. Not built
18/row	270, ~3 s to build alone	one field per hex pair aligns perfectly and cannot be paid for

The escalation if columns come out ragged within a block is 7/row, not 18.

The key-presence poll costs 340 us a tick

Measured on 1.7.17a over 200 calls each, with a key in the slot:

call	cost
`file.usbdriveexists()`	237 us (see 'A key pulled from under a running app')
`ui_usb_btns()`	297 us -- the read, plus the change test
`ui_tick()`	340 us, measured whole and not summed from the parts

So 2 Hz is **0.07 %** of the instrument, and fractional intervals are honoured by `display.OBJ_TIMER`. Measure the tick itself rather than its parts, and know which tree answered: a tick that also calls `ulog.resume()` costs **629 us** against the 340, which is the whole reason it does not, and a load that fails silently leaves the previous app running while answering every query cheerfully.

Raising the rate cannot make a key appear sooner. Observed on the panel: pulling the key hides the buttons at once, every time, while an insertion takes about **2 s** to bring them back, whether or not a button is pressed in the meantime. Both directions run the same code, so the delay is `file.usbdriveexists()` answering 0 until the key is mounted. 2 Hz already samples far faster than that signal moves. `tools/probe_keylatency.py` holds the measurements and the two ways of taking them that cost a sluggish panel and a power cycle.

The 111 us quoted for `usbdriveexists` elsewhere was taken with the slot **empty** -- a different measurement, not a disagreement. The expensive case is the one every per-line and per-row gate pays.

Clearing the handler is the only timer off switch

Measured on 1.7.17a against a 1 s timer, counting handler calls over 4 s of idle:

state	calls
enabled	4
`display.STATE_INVISIBLE`	4 -- an invisible timer still fires
handler set to `''`	0
handler restored	4
another screen in front	0

So the state call every other control on this panel uses does not work here. **The last row does not cover the options form**: it was taken with the firmware's own HOME screen in front, the app backgrounded, and `sdec.optscr` is a sibling screen of our own with the app still foreground, which was never measured. Observed with the form up: the edit fields need several presses over 5-10 s to open, intermittently, while the four buttons stay responsive -- which is what a 2 Hz poll competing with a modal editor looks like, and not what a stopped timer looks like. Hence the explicit `ui_tick_off()` in `sdec.options()`, correct whichever way the firmware behaves.

A tick is dispatched like a press, so a run that holds the interpreter **defers** ticks rather than suppressing them: measured, a 10 s busy hold releases 2, so the backlog is shallow.

ui_latency_s = 0.5 and ck_press_budget_s = 8

No running Lua can observe a front-panel press -- not `display.setevent` (the firmware cannot call the handler while the interpreter is busy, so presses queue), not `display.waitevent` (it ignores a zero timeout and blocks indefinitely, wedging the instrument until a LAN ABORT), not `delay()`. Only a handler **returning** lets a press through, so responsiveness is a property of how long each handler runs and the budget is not a poll interval.

The slowdown from chunking is repaint_cost / budget, so the budget must be ~10x a repaint to stay under 10 % overhead. Decode throughput is ~**881 byte/s** for samples already in Lua (20 000 samples = 0.236 s: 0.115 s of edge detection plus 0.120 s of framing, yielding 208 bytes at 8 samples/bit), so 500 ms is about 440 bytes of work -- more than the 240-byte page and the 256-byte chunk, and less than a human's threshold for "did that button do anything?".

Do not budget anything else against the 881. It excludes reading the samples out of the capture buffer, which is the larger half at 21.7 us a sample. End to end the figure is ~**358 byte/s**, and ~**250 byte/s** at the streaming modes' 4.17 samples/bit, so anything sized against 881 is 2.5-3.5x optimistic. `ck_win_us = 48` is the per-sample figure that includes the read.

`ck_press_budget_s = 8` is deliberately much larger, a press that returns having been answered: 8 s gives **27 windows a press** -- a full 4 076 503-reading buffer in 25 presses, a 30 s recording at 4800 Bd in 4 -- and stays inside the 12 s bound `bench_panel.py` holds capture presses to, with room for the final window to run past the check that admitted it. The panel does not repaint during those 8 s and a press inside them is queued, not lost. FRAME mode makes the same deal for up to 9 s.

At the measured ~250-360 byte/s, 8 kB is ~30 s of decoding and 32 kB about two minutes, which is what the choice between the two recording sizes is really about.

The S/N cliff is at 12 dB

Swept offline at 9600 Bd over 12 capture phases per point, injecting uniform noise as a fraction of the swing: byte-exact **12/12 at every level down to 45 % of the swing**, which reports 12.5 dB, and at 50 % `sig_levels` **refuses all twelve**. So the band boundary sits one step above a refusal, and what lies past it is 'no clear logic levels' rather than a wrong answer.

The same three dB figures at 3.3 V, 1.0 V and 0.5 V of swing, to 0.1 dB. That is what justifies one pair of thresholds for every line the app will see: the tolerance is a fraction of the swing, so the dB at which it fails does not move with the swing.

The 40 % noise tolerance quoted elsewhere is a **peak** figure and this cell is an **rms** one, so 20*log10(1/0.4) = 8 dB is not the cliff in these units and nothing should be derived that way. `gen_serial`'s `noise` is the peak of a uniform draw, whose rms is 1/sqrt(3) of it: 40 % peak is 23 % rms, which is 12.7 dB.

`ui_sn_bad = 15` sits **above** the cliff, not on it, because red has to mean "close to failing" rather than "failing": 15 dB is noise at ~31 % of the swing, which still decodes 12/12. `ui_sn_good = 30` is noise under ~5.5 %.

Why the FIT bands are 0.90 and 0.72

0.90 is `sdec.autolock_fitq`, the quality the app itself demands before it will pin a rate, so green means "good enough to autolock".

0.72 sits 10 % above a structural knee at 0.65: `sig_fit`'s quality is (1 - mean_err/T) * (nfit/nsmall) and only widths within 0.35*T of a multiple count, so tightness alone cannot fall below 0.65 and anything under it means **coverage** is collapsing. At 0.72 either the mean width error is a quarter of a bit time or a quarter of the widths match no multiple, and both are close to losing bits.

The options form vertical geometry

The edit-object height is measured, off `docs/img/options.png`: a field created at y draws its box from y + 4 to y + 53, so it is **50 px** tall. Arithmetic built on a 38 px guess reported 8 px of clearance over an overlap.

Against a button row at the shipped `ui_opt_btn_y = 360`, with the clamp in `build_options()` as written:

fields	columns	rows	pitch	gap	clear of the buttons
6	2	3	100	50 px	73 px
8	2	4	99	49 px	5 px (the UART-only build)
9	2	5	74	24 px	5 px (a protocol module loaded)

An eighth field is free -- seven already needs four rows and leaves the second column's last slot empty. The ninth forces a fifth row. A tenth also fits in five rows; an **eleventh** forces six and is where this stops working.

A gap is a touch target, not decoration. Reported from the bench at the old 4 px gap: the fields need several presses over 5-10 s to open, intermittently, while the four buttons -- a row with nothing above or below them -- stay responsive, and the last field, the only one with no neighbour beneath it, is responsive too. That is the signature of an ambiguous boundary, where which object receives a touch is the firmware's hit-test order rather than the operator's aim. 24 px is six times that gap, and a 50 px gap between 50 px boxes means a touch can miss by 20 px and still land right.

`ui_opt_btn_y` is 360 and not 366 because a button is **60 px** tall on this screen, not the 58 the main screen assumes: at 366 the row occupies 367..426 of a 431 px content area, and 5 px from the bottom edge reads as clipped. 360 gives 11 px, matching the main screen's row, and still leaves 29 px under the last field. The ceiling on the pitch is the button row, so raising `ui_opt_dy` buys nothing; moving the buttons down is what it would cost.

The options form field and button widths

`display.create` takes **no width** for an OBJ_EDIT_*: the signature is (parent, kind, x, y, label, help), so the firmware sizes the widget and the app places only the origin. The reference's own example puts an OBJ_EDIT_NUMBER at x = 400 and an OBJ_EDIT_CHECK at x = 200 on the same row, so the widget is narrower than half the screen and two columns are possible. The widget width itself is **not measured**, so the second column's x is the one number to check on the glass.

`ui_opt_px = 96` bounds an option-field entry. A character count cannot decide it: 'In + FC Out' fits at 11 characters while 'TRIGGER key' fades at 11, because seven of its letters are uppercase. Entries are weighed with the header cells' two advances -- 10 px uppercase, 8 px otherwise -- against a bound measured between the two: **'In + FC Out' at 96 px renders whole and 'TRIGGER key' at 102 px does not.** Past the bound the firmware fades the tail rather than clipping, so an over-long entry is both unreadable and mistakable for a shorter one in the same field ('Trig In+FC Out' draws as 'Trig In+FC', beside a real 'Trig In').

Eleven characters is the field's limit in practice: measured on the panel, the value column is 146 px inside its box and the face advances ~10.6 px.

The button row spans the screen, with widths taken from the space available rather than from the len * 14 + 8 model -- sized to the model the labels sit nearly edge to edge inside their faces, which is legible in a screenshot and tight on the glass. 22 px margins, 12 px gaps, the two long labels given the extra: 22 + 200 + 12 + 220 + 12 + 150 + 12 + 150 = 778, so the right margin matches the left. Every label clears the rule that sizes it by 30 px or more: 'Lock Detected' wants 190 and has 220, 'Auto Detect' 162 in 200, 'Apply' 78 in 150, 'Cancel' 92 in 150. A uniform 150 truncates both long labels.

The armnote cell goes through ui_set

`armnote` writes `ui_log_t`, the status row's right-hand cell, because that is already where the panel puts what the operator can DO about a long operation. The left cell is a composite built by `ui_refresh_do()` and `sdec.ui_status` is a mode WORD asserted against elsewhere, so neither is borrowed for a sentence. One cell has news, so one cell is written: a full refresh here would run over a mid-capture state that `clear_result()` has already zeroed, painting blanks over the previous result.

Through `ui_set`, which is what makes the note temporary. `ui_set` keeps the last text written per handle and skips a rewrite that would change nothing. A bare `settext` leaves that cache holding whatever the previous refresh put there, so the next refresh after a failed arm found its own stale entry, decided the cell already said the right thing, and the wait note **stuck on the panel for the rest of the session** -- the exact inverse of a note that exists to say "right now".

It promises the TRIGGER key only when the key is wired in. `keyed` is `sdec.armkeyed`, which `arm_source()` sets only where the key really was blended into the arm, and three live cases are not: Arm key off, a firmware whose blender refuses, and an anchored rear-BNC capture. Printed unconditionally, the cell told the operator their one escape worked at the exact moment it did not, for up to two minutes.

Why BAUD, FORMAT and IDLE fall back to forced values

Those three are the only header cells the options form can force, and the streaming modes hand back a file rather than an `sdec.res`, so without the fallback all three read '--' for a whole run. A streaming mode cannot be entered without a locked rate, which makes them the three cells that are certain; the measured cells (LOGIC, THRESH, SA/BIT, FIT) do stay '--', nothing having been fitted.

`%.0f` on both, never `%d`. A locked rate can be a MEASUREMENT -- `decode_from` copies `force_baud` into `sdec.baud` with `snapped = true`, and 153588.155 has been on this panel -- and this is the repaint path for every capture. Lua 5.0.2 truncates a fraction silently where 5.4 raises, so a `%d` raises the whole refresh on a modern host and reaches the BAUD cell disagreeing with `baud_text()` beside it on the instrument.

How far a misaligned head reaches

`ui_head_extent` is one function because three indicators share its number and must agree: the note names the extent, ERR counts it (`ua_err_count`), and the hex rows covering it are red. Asked separately, the colour asked the narrower question -- did this byte FAIL a check -- so a head was reddened only where it happened to fail: 'the first 35 bytes are misaligned' beside ERR 35, with bytes 17-32 painted white for passing their stop bit by luck.

It reports damage, not suspicion. `headsusp` is `idle1 - 1`, the bytes before the first inter-message gap, which `ua_run` excludes from the parity vote as a conservative region. It is not a count of bad bytes, and reported as one it over-claimed badly: **'the first 79 byte(s) are misaligned' over a capture with 3 flagged bytes** and a first row that was not even red, the fox vector running 79 bytes before its first gap. An earlier capture said 36 against 20 errors, close enough to look plausible, which is how it survived. So the number is the last flagged byte inside that region, and only countable when the head is ON SCREEN -- a 32 kB tail does not carry its own head's errs, and `ck_tot.headbad` is the only figure for that case.

Why the note is worth a line at all. On a continuously busy line the trigger fires wherever it fires, and the first edge carries no evidence of how long the line was at mark, so the framer can anchor inside a byte and stay wrong until the first idle. The bytes are THERE, on screen, looking like data; most are flagged, but a frame can pass its stop bit and parity by chance.

Why the capture-FULL note excludes a retained tail

A capture that comes back exactly full looks identical to a complete short message, and on a continuously busy line -- a micro dumping debug text, which is the main use of this app -- every capture is full. The note names the remedy too: locking the detected rate drops the sample rate to suit it and lengthens the window, 19 bytes to 192 at 9600.

Not for a recording's retained tail, and this is the guard that was missing. After an 8 kB run the result on screen is a SLICE of a capture that went to a file, so 'the capture was full' is not a fact about it, and the figure came out as the FRAME window at the recording's own sample rate -- **423 bytes, measured on the panel**, a number belonging to no capture that had happened. The remedy is wrong there too: a recording mode requires a locked rate, so 'lock the baud rate for a longer window' names a thing already done. `r.first` is the tail marker: `ck_tail_result()` sets it and nothing else in the app does, so a result carrying it is by construction a window onto something larger.

Why a forced width assumes no parity

`decode_from`'s forced path takes `sdec.force_par or PAR_NONE`, because forcing a width bypasses the format search entirely -- which is the point of forcing on a bad wire, the search being the stage impulse noise captures. So Data Bits = 8 with Parity = Auto decodes as 8N1 without ever trying 8E1 or 8O1. The header does show "8N1", but a reader who chose "Auto" would fairly take that as DETECTED rather than ASSUMED, hence the note. MIDI and LIN set both fields and Lock Detected sets parity from the result, so it fires only for a hand-forced width.

Deleting buttons from a dense list

`sdec.ui_btn` is POSITIONAL and can have a hole: `ui_build` stores the three right-margin buttons at `ui_btn[7..9]` before the bottom row fills 1..6, so a bottom row that partly fails leaves a gap, and `table.getn` stops at a gap -- a walk over `ui_btn` would never reach 7, 8 or 9. Those would be dropped unreferenced when `ui_btn` is nil'd, surviving only by the firmware's parent cascade, which this app never relies on.

`ui_allbtn` is appended to in creation order, so it is dense whatever fails. Delete from it INSTEAD of from `ui_btn`, not as well: `del_obj` on an already-deleted handle is refused, which counts against `delfails` and strands the handle on `sdec.orphans`, and the rebuild guard reads both.

The display object pool fails silently

`display.create` returns nil when the pool is exhausted, posts nothing, and does not raise. Event **1701** is logged the first time only; after that every child is created against a nil parent and `start()`'s pcall sees success -- which showed as a screen with no buttons and no message. So `ui_build` raises on a nil screen, reusing `start()`'s existing teardown-and-report path, and checks the button count afterwards: the screen existing means the first guard passed, and a pool that ran out PART WAY through still leaves nothing to press. The buttons are built last precisely so that check covers everything before them.

Every refused `del_obj` goes on `sdec.orphans` rather than being dropped, because a teardown nils its variables afterwards and dropping a refused handle loses the only reference to a live firmware object -- stranded until a power cycle. That accumulation is what raises 1701.

Rects, the padlock and the progress bar as objects

No rule at `ui_vr3_x` closing the ASCII column on the right: the ASCII text is arbitrary bytes, so its rendered width is not knowable in advance -- mixed case measures 7.80 px/char but a row of 'W's does not -- and a boundary the content can cross is worse than no boundary. Nothing follows the gutter. The two rules that do exist are kept in their own array rather than in `ui_rule`, because unlike every other rule they are recoloured per refresh.

The padlock is drawn from RECTS because the panel font has no padlock glyph: TSP strings are bytes the firmware renders from its own proportional face, and U+1F512 is not in it. Four objects -- a solid body and a three-piece arch -- which is worth counting, object creation being the destabilising operation behind the one-build-per-power-cycle rule. Recolouring them afterwards is not. The glyph sits after the WIDEST value the BAUD cell can hold rather than the one currently shown, and its colour changes through `sdec.ui_fill`, which counts refusals in `fillfails` so that a setfill that turns out not to work shows in the status row instead of the icon lying.

No hit rect and no setevent on any of them. EVENT_PRESS does not reach an OBJ_RECT -- "1717 Attribute event, does not apply to an object of type Rectangle" -- and the refusal is severity 2, which raises an error dialog OVER the panel. An OBJ_RECT also draws an OUTLINE whether or not it is filled, so a hit box paints a border round its field brighter than any rule in the header table. That is why Lock Rate is a button and why Options > Rear BNC is the trigger cell's control.

display.OBJ_TIMER is the only periodic work

`display.OBJ_TIMER` is a display OBJECT with an interval and a repeat count, and it reports through EVENT_PRESS like a button -- which is why no EVENT_TIMER constant exists and why probing for one finds nothing. Taken from the instrument's own clockIV3 sample, which drives an analog second hand this way. **TIMER_FOREVER is 0 on this firmware, measured**; it is the repeat count, not a flag.

Nothing else can do it. A Lua loop cannot: the firmware dispatches a handler only while the interpreter is idle, so a polling loop never returns and every button on the panel goes dead. And `display.waitevent()` waits on a `display.prompt()` dialog rather than on these objects, ignores a zero timeout, and wedges the instrument until a LAN ABORT recovers it.

A tick cannot interrupt a capture: it is dispatched like a press, so it waits for the handler to return. `sdec.capture()` turns it off for the duration anyway, and `ui_tick()` is a cheap no-op while the app is busy, a deferred tick having to not repaint objects a run is still using.

The six bottom buttons and the three in the margin

Six buttons across 8..774, 84 to 120 px each, all far above a comfortable touch target at 58 px tall. `View` and `Mode` are STATIC labels rather than naming the state they cycle to: there are four views and three modes, which one is showing is named in the status row and the MODE cell, and a button that relabels itself has to be read twice. `NewLog` is a button rather than an options field because separating one device's messages from the next is something done WHILE watching them arrive, and a control you must leave the screen to reach is a control you do not use. `Save` stays even though frame mode logs every capture, because the two write different things: the auto-log is one message per capture as text and hex, where Save writes the full report.

Up, Lock Rate, Dn, in that vertical order, so neither page button needs reading. A paging control is not optional: 240 bytes is one page only in FRAME mode, where a 32 kB streaming capture is 137 pages, and without these the app collects 32768 bytes and shows the first 240. Lock Rate is here because the padlock glyph cannot be pressed, which would otherwise leave Options > Lock Detected as the only route to the

most useful action on the panel. All three hide themselves when they have nothing to do -- the page buttons when the capture fits one page, Lock Rate when there is no detected rate to pin -- which is why they are tracked apart from the bottom row.

Right edges all at 785, each face only as wide as its own label, because width these buttons do not use is width the dump rows get: a TEXT row is 80 columns of a proportional font, needing 636 px against the 592 it had at 185, so the right-hand characters were drawn under Page Up -- and the page arithmetic still counted them, so paging stepped over bytes that were never visible. 'Up' and 'Dn' rather than 'Page Up' and 'Page Dn': a face needs len * 14 + 8 px, so two characters fit in 36 and the 56 given is a comfortable target with room over.

Narrowing them does not widen the TEXT view by itself. The dump's usable width is set by the LEFTMOST margin button, and that is Lock Rate at 645 -- 'Lock Rate' measures 134 px and cannot follow the page buttons down. Rows 7..10 sit under it, so `cols` stays at 80 (631 px, ending at 639) until Lock Rate itself moves. `tools/render_png.py` measures every row against that edge.

Lock Rate is shown only in the 'auto' state. 'locked' has nothing for it to do, so it hides rather than offering a press that would UNLOCK -- a button labelled 'Lock Rate' must not unlock -- and with nothing decoded there is no rate to pin. Unlocking is Options > Auto Detect, which is also the press that re-detects: the two belong together, and 'not locked' is not a thing to aim for on its own. With auto-lock on the button is usually absent, which is correct; it is the manual route for the captures auto-lock DECLINED -- too few frames, a poor fit, an unstable baseline.

The page counter is four objects on the note row

Paging blind is the complaint it answers: the buttons moved the view but nothing said how much there was or where in it you were. It sits on the NOTE ROW rather than in the right margin between the page buttons, which is the wrong home twice over -- it puts a sentence-sized fact in a 44 px gap, and it spends margin width the dump rows want.

Four objects, because a text object carries ONE colour and this line carries two: the numerals at full white, 'page' and 'of' at 66 %. They are created with their colours and never recoloured, so the counter needs no colour shadow cache -- one fewer entry `ui_invalidate` could be missing. The numerals are JUST_RIGHT in fixed columns so the digits land in the same place at 7 and at 137, and the words are left-anchored: nothing moves as the page changes, which is the point.

The visibility shadow caches

`ui_pgvis`, `ui_lockvis` and `ui_savevis` are caches, where nil means "unknown, so write it on the next refresh". `sdec.ui_lockbtn` is NOT: it is the button's handle, assigned in the create loop. Clearing it leaves it nil for the life of the app, which makes the visibility block in `ui_page_btns()` (`if sdec.ui_lockbtn ~= nil`) dead code -- Lock Rate then keeps whatever state `create()` left it in, is never hidden, and sits on the glass with the rate already locked, offering a press it must not offer. `bench_panel.py` measures the margin buttons by pixel and is what catches it.

Why the vertical rules follow the view alone

They belong to the hex view and are hidden in the others, or two hairlines cut across an 80-column TEXT row that has no columns to divide. MIDI and LIN are hidden for the same reason and not as an oversight: both are `cols = 1`, one message or frame of free text per row.

The view decides, and only the view. The capture mode has no say: in the 8 kB and 32 kB modes the table structure is the same table, and a grid that appears and disappears as the Mode button is pressed reads as a fault rather than as information. Testing the mode's own `view` flag as well left the rules missing for the whole of a recording mode until a capture had finished -- which is exactly when an operator is looking at the panel and deciding whether it is working.

They are hidden by painting them `ui_c_hide`, which matches the background, and NOT 0: the firmware reads a fill of 0 as "unset" and paints nothing at all, so `setfill(h, 0)` is a refused write rather than a black rectangle. `tools/mock_display` raises on it by name. A background-matching colour is an ordinary fill, so hiding uses exactly the mechanism showing uses and cannot fail differently.

The rear BNC cell states and the uncertainty mark

The cell is EMPTY when the connector is off: a marker that is always present is not a marker, and a permanent amber 'EXT TRIG' over an Options page reading Rear BNC = Off is the panel asserting hardware that is not in use. The direction is the fact the cell exists to carry, Rear BNC being two independent booleans -- `trigext`, an input that waits for an edge, and `fc_out`, an output that pulses a credit per capture -- which 'EXT TRIG' named neither of:

state	text	colour
`trigext`	TRIG IN	green
`fc_out`	TRIG OUT	amber
both	TRIG I/O	cyan
neither	(empty)	--

The '?' is the app's own uncertainty mark, the one the BAUD cell uses for an unlocked rate. A rear INPUT is configured-but-not-in-force under Free run, because free run means DO NOT WAIT, and that is the case that misleads: the form still shows it set. The colour stays the mode's, the three being spoken for, and `ui_notes` carries the sentence.

The trigger key does not cancel a panel-initiated run

While streaming, the status row's right-hand cell carries the CONTROLS: an operator watching a four-minute capture needs to know how to get out of it more than they need the debug log's line count, and both will not fit in the left string, which already ends near 385 px of the 430 available.

Which control is true is decided by the state, and for a one-press run the true answer is none. 'TRIGGER key = stop' on the strength of `cancel_setup()`'s blender latch is FALSE, measured: pressed 20 % into a 32 kB decode, with tens of seconds of polling still to come, the run finishes 'full' and the latch is EMPTY -- the key's event never reaches the blender while a panel-initiated run executes. The plumbing behind it is sound (a firmware trigger timer blended in as stimulus 2 DID cancel at 8 s, keeping 497 bytes), so what is false is only the claim that a finger can deliver it. A touch press cannot act either: presses queue.

Painted once, before the run, and correct for its whole duration: `sdec.progress()` repaints only the counter cell, so nothing overwrites that cell until the run returns. `strm_inflight` is what makes it possible -- it is set by `capture()` before the refresh, because `ck_running` does not become true until `stream_begin()` is already inside the job.

'Stop', not 'Exit': Mode aborts the RUN without changing the mode, so a label promising an exit describes behaviour the button does not have. And no 'Mode=...' hint at all in the resting case: Mode CYCLES, so 'Mode=Exit to FRAME' is wrong in two of the three states that show this cell, and naming the destination

correctly still reads as the mode you are IN, under a MODE cell that means exactly that.

Why flog_status asks about the key before the filename

No key outranks both the filename and the promise. A path allocated while a key was in the slot stays set after it leaves, and `flog_why` is nil because nothing has failed -- nothing has been ATTEMPTED. So the cell went on reading 'log: bytes248 2721 B' with the key in a pocket: the bytes are real and on that key, and the row still reads as logging in progress. Seen on the instrument.

The same test covers the empty case, where 'on first capture' is a promise the app cannot keep: `mode_exit()` clears `flog_why` after every recording, so the `flog_why` branch above it cannot carry that one.

The window counter counts from window 1

Under flow control a press is a whole transmission, so 'recording... 40 %' alone reads as one 32 kB window that is taking a very long time; the window number is what says the run is making progress through a message larger than one buffer.

`fc_win` is incremented after each window RETURNS, so it is 0 throughout the first one -- and a `>= 1` guard therefore suppressed the counter for exactly that window and made 'win 2/32' the first thing the operator ever saw, reading as a run that had already lost its beginning. `(fc_win or 0) + 1` is the correct 1-based number from the start.

Who raises and who clears the loading cover

Creating a screen makes it the active one, which is why the operator can otherwise watch this app build itself: `ui_build()` creates the main screen and fills it, `build_options()` then creates the options screen -- bringing THAT to the front -- and fills it too, and only afterwards does `start()` call `changescreen(ui_scr)`. Two screens assemble in full view, in the wrong order, ending on a jump. At ~12 ms a text object and ~134 objects a launch, there is well over a second of drawing to hide.

So the cover is raised after EVERY screen create, not once at the start, and `ui_loading_raise()` is idempotent so the call sites do not have to care. **Who raised it decides who clears it**: during a launch `ui_build()` raises it and `start()` holds it, so `build_options()` only takes the front back; pressed by hand on a session whose form was never built, there is no cover and `build_options()` raises its own. Ownership is "did I create it" rather than a flag, because the two paths differ in who called first, not in what they want.

The options screen is the one that looks like a fault rather than slowness: the app appears to open its settings page at launch and then leave it.

Never fatal. The pool can be exhausted and `display.create` returns nil silently, so a missing cover means a visible build -- the behaviour this replaces, not a failure -- and every call here is guarded with an advisory return.

Why the Protocol field comes first and is spelled out

Protocol overrides the fields below it: MIDI is 31250 baud 8N1 by specification, so choosing it fills in the wire parameters instead of asking for them, and switches the view to the parsed message list. LIN pins 8N1 and normal polarity the same way but deliberately NOT the baud rate -- LIN specifies a RANGE, 1 to 20 kBd, so there is nothing to pin and the auto-detector remains the best answer. Anyone holding the bus datasheet can still type it in.

Spelled out rather than `unpack()`'d. The instrument's Lua 5.0.2 has a global `unpack()`, but the offline harness runs on a modern Lua where it moved to `table.unpack`, so the clever form works on the DMM and raises in every test. There are exactly three possible protocols, so a branch per count is shorter than the portability shim. `pl.n == 1` creates nothing and `sdec.opt_proto` stays nil, which every reader already copes with, `create()` returning nil on refusal being normal here.

Why the 10 V range is the only one offered

`sdec.range` is fixed at 10 V because it is also the best one: its digitize bandwidth is **440 kHz against 210 kHz on 1 V and 100 mV, and 17 kHz on 100 V and 1000 V** -- and 17 kHz would cap decoding near 30 kBd while looking like a decoder fault rather than a front-end one. Nothing worth decoding swings under 1 V, and anything over 10 V wants a divider (`docs/lin-divider-panel.png` shows one), which is a better answer than a range that throws the bandwidth away.

The Trigger field therefore offers three sources and not four: the rear BNC has its own field, which says what the back panel does alongside the primary source. Its entries come from `sdec.ui_trigsrc` rather than from literals, because the note row's left cell shows this field's value and the two must be the same words.

Why one field carries both rear BNCs

Two orthogonal booleans (`trigext` and `fc_out`) become four entries in one field, because the form has no room for another field -- see 'The options form vertical geometry' -- and an option field's entries cost no vertical space at all. It is a fair grouping rather than a squeeze: both settings configure the REAR BNCs, one the input and one the output, and neither is a trigger SOURCE.

Off by default, and that remains a requirement rather than a courtesy: this is a serial decoder and it must work with one probe on one line and nothing else attached. A default of anything else would make an unconnected rear connector part of the critical path, and for FC Out it would also mean pulsing a BNC that may be wired to other bench equipment.